
*Geração Automática de Código LaTeX a Partir de Expressões
Matemáticas Manuscritas*

Emanuel Henrique Farias

Emanuel Henrique Farias

*Geração Automática de Código LaTeX a Partir de Expressões
Matemáticas Manuscritas*

Relatório de Trabalho de Conclusão de Curso (TCC)
apresentado ao Curso de Graduação em Ciência da
Computação, da Universidade do Estado de Santa Ca-
tarina (UDESC), como requisito parcial da disciplina
de Trabalho de Conclusão de Curso.

Orientador: André Tavares da Silva - Doutor (orientador)

Joinville

2016

Emanuel Henrique Farias

*Geração Automática de Código LaTeX a Partir de Expressões
Matemáticas Manuscritas*

Relatório de Trabalho de Conclusão de Curso (TCC)
apresentado ao Curso de Ciência da Computação da
UDESC, como requisito parcial para a obtenção do
grau de BACHAREL em Ciência da Computação.

Aprovado em 28 de Novembro de 2016

BANCA EXAMINADORA

André Tavares da Silva - Doutor (orientador)

Gilmário Barbosa dos Santos - Doutor

Lilian Berton - Doutora

Rafael Stubs Parpinelli - Doutor

Resumo

Escrever expressões matemáticas complexas no computador é uma tarefa difícil e demorada. As pessoas estão habituadas a escrever expressões matemáticas com papel e caneta, seria mais natural se elas continuassem escrevendo dessa forma para que em seguida a imagem da fórmula seja convertida automaticamente pelo computador em código LaTeX correspondente. Sistemas de reconhecimento de expressões matemáticas possuem várias utilidades desde a criação de códigos para documentos no formato LaTeX até auxiliar portadores de deficiência visual pela leitura de papéis com expressões matemáticas. Este trabalho de conclusão de curso teve como objetivo estudar os métodos utilizados na literatura e desenvolver um sistema de reconhecimento de expressões matemáticas escritas à mão, implementando uma aplicação *web* capaz de gerar códigos LaTeX. Para desenvolver o reconhecedor de expressões matemáticas, foram implementadas as seguintes etapas: pré-processamento, segmentação, classificação e análise estrutural. Na etapa de pré-processamento, são aplicados filtros de mediana e limiarização para gerar uma imagem binária. Para a segmentação, foi aplicado o método de componentes conexos. Antes de classificar, os símbolos foram dilatados e redimensionados. Para a classificação, foi utilizada uma rede neural convolucional. Após classificados os símbolos, é realizada a análise estrutural, utilizada para construir uma estrutura de dados que representa a expressão matemática. O código LaTeX foi gerado percorrendo esta estrutura em árvore gerando a *string* da expressão matemática. Para validação do sistema, desenvolvido como aplicação *web*, foram coletados dados de seis pessoas e com isso foi possível observar pontos positivos e negativos em cada uma das etapas do trabalho. Este foi um trabalho completo com o desenvolvimento de todas as etapas de um reconhecedor *offline* de expressões matemáticas. Na literatura, não foi encontrado trabalhos que abordam todas as etapas de um reconhecedor *offline* manuscrito, e por isso, este trabalho tem a sua relevância. Mesmo utilizando métodos tradicionais na área de processamento de imagens, este trabalho conseguiu alcançar bons resultados.

Palavras-chave: *Reconhecimento Offline, Reconhecimento Manuscrito, Expressões Matemáticas, Aprendizado de Máquina, Reconhecimento de Padrões.*

Abstract

Write complex mathematical expression in the computer is a difficult and time consuming task. People are used to write mathematical expressions using paper and pen, it would be more natural if they continue to write in this way so that the image of the formula will automatically be converted by the computer corresponding LaTeX code. Mathematical expressions recognition systems have several uses from building codes to documents in LaTeX format to assist the visually impaired by reading papers with mathematical expressions. This paper aims to study the methods used in the literature and develop a handwritten mathematical expression recognition system, implementing a web application that can generate LaTeX code. To develop the mathematical expression recognizer, the following steps were implemented: preprocessing, segmentation, classification and structural analysis. In the preprocessing, median filter and thresholding are applied to generate a binary image. Before classifying, the symbols were dilated and resized. For the classification, a convolution neural network was used to recognize each symbol. After classification the structural analysis is used to build the data structure that represent the math expression. The LaTeX code was generated by traversing the tree structure creating the math expression string. To validate the system, developed as a web application, data was collected from six people and was possible to find the downsides and strength of each stages. This was a complete work developing all the stages of a off-line mathematical expression recognition system. Even using traditional methods in the image processing area, this work has achieved good results.

Palavras-chave: *Offline Recognition, Handwriting Recognition, Mathematical Expression, Machine Learning, Pattern Recognition.*

Conteúdo

Lista de Abreviaturas	6
1 Introdução	10
1.1 Objetivo	12
1.2 Objetivos Específicos	12
1.3 Metodologia	13
1.4 Estrutura do Trabalho	13
2 Fundamentação Teórica	14
2.1 Processamento e reconhecimento de padrões em imagens	14
2.2 Digitalização	15
2.3 Visão geral do reconhecimento de expressões matemáticas	15
2.4 Reconhecimento <i>online</i> vs <i>offline</i>	16
2.5 Morfologia Matemática	17
2.5.1 Erosão e Dilatação	18
2.5.2 Abertura e Fechamento	19
2.6 Pré-processamento da imagem	19
2.6.1 Filtragem	20
2.6.2 Limiarização	21
2.7 Segmentação	21
2.7.1 Vizinhança	22
2.7.2 Rotulação de Componentes Conexos	23
2.8 Extração de Características	23

2.9	Aprendizado de Máquina	24
2.9.1	K-Vizinhos mais próximos	25
2.9.2	Máquina de vetores de suporte	26
2.9.3	Redes Neurais Artificiais	27
2.10	Análise estrutural	31
2.10.1	Relação espacial e atributos dos símbolos	31
2.10.2	Aspecto Comportamental	33
2.10.3	Dominância	33
2.11	Grafos	35
2.12	Árvore Geradora Mínima	35
3	Trabalhos Relacionados	37
3.1	Pré-processamento	38
3.2	Segmentação	39
3.3	Extração de Características	40
3.4	Métodos de Classificação	41
3.4.1	Redes Neurais Convolucionais	42
3.5	Geração da Árvore de Expressões Matemáticas	44
3.5.1	<i>Baseline Structure Tree</i>	45
3.5.2	Árvore Geradora Mínima	48
3.6	Geração do LaTeX	51
3.7	Considerações	51
4	Geração Automática de Código LaTeX a Partir de Expressões Matemáticas Ma- nuscritas	53
4.1	Pré-processamento	54
4.2	Segmentação	55
4.3	Pós-segmentação	56

4.4	Reconhecimento	56
4.5	Análise Estrutural	59
4.5.1	Identificação das barras horizontais	60
4.5.2	Grafo e Árvore Geradora Mínima	61
4.5.3	Árvore de <i>Baselines</i>	63
4.5.4	Conversão para código LaTeX	65
4.6	Escopo	65
4.7	Implementação	66
5	Resultados	69
5.1	Expressões reconhecidas	69
5.2	Problemas Encontrados	70
6	Considerações Finais	74
6.1	Trabalhos Futuros	75
	Bibliografia	77
A	Apêndice: <i>Datasets</i>	82
B	Apêndice: <i>Pseudocódigos</i>	85
B.1	Pós-segmentação	85
B.2	Indentificação das Barras Horizontais	86
B.3	Associa Baseline pela MST	86

Lista de Abreviaturas

ADAM	<i>Adaptive Moment Estimation</i>
AM	Aprendizado de Máquina
BST	<i>Baseline Structure Tree</i>
CNN	<i>Convolutional Neural Network</i> ou Redes Neurais Convolucionais
DFT	<i>Discrete Fourier Transform</i> ou Transformada Discreta de Fourier
EE	Elemento Estruturante
GD	<i>Gradient Descent</i> ou Gradiente Descendente
GPU	<i>Graphics Processing Unit</i> ou Unidade de Processamento Gráfico
k-NN	<i>k-Nearest Neighbour</i> ou k-vizinhos mais próximos
MLP	<i>MultiLayer Perceptron</i> ou Perceptron de Múltiplas Camadas
MST	<i>Minimum Spanning Tree</i> ou Árvore Geradora Mínima
NCC	<i>Normalized Chain Code</i>
OCR	<i>Optical Character Recognition</i> ou Reconhecimento Óptico de Caracteres
PDA	<i>Personal digital assistant</i> ou Assistente Pessoal Digital
PNG	<i>Portable Network Graphics</i>
ReLU	<i>Rectified Linear Unit</i>
RNAs	Redes Neurais Artificiais
SGD	<i>Stochastic Gradient Descent</i> ou Gradiente Descendente Estocástico
SVM	<i>Support Vector Machine</i> ou Máquina de Vetores de Suporte
TXL	<i>Turing eXtender Language</i>

Lista de Figuras

2.1	Etapas básicas do reconhecimento de expressões matemáticas.	15
2.2	Reconhecimento <i>offline</i> vs <i>online</i>	16
2.3	Imagem de dois EE com seus centros na origem.	17
2.4	Dilatação à esquerda e Erosão à direita.	19
2.5	Expressão antes e depois de segmentada	22
2.6	Vizinhança-4 à esquerda e vizinhança-8 à direita.	22
2.7	Modelos de Aprendizado de Máquina.	25
2.8	k-NN com k=3.	26
2.9	Classificador SVM encontrando o melhor hiperplano.	27
2.10	Modelo de RNA com um neurônio Perceptron.	28
2.11	Modelo MLP.	29
2.12	Arquitetura de uma CNN.	31
2.13	Grafia dos símbolos, centroides e limiares	32
2.14	Possíveis regiões de alcance de um símbolo.	34
2.15	Regiões de abrangência de três classes comportamentais.	34
3.1	<i>Skew correction</i>	39
3.2	Regiões de alcance do símbolo e limiares do centroide.	45
3.3	<i>Baseline</i> principal ao centro.	46
3.4	Método BST aplicado em expressões manuscritas.	48
3.5	Irregularidades do <i>layout</i> horizontal.	49
3.6	<i>Baseline</i> dominante construída.	50
4.1	Imagem de entrada (fornecida pelo usuário) e a saída que é mostrada no navegador.	53

4.2	Ferramenta de seleção e corte.	54
4.3	Etapas do reconhecedor de expressões matemáticas manuscritas.	54
4.4	Imagem de entrada após a binarização.	55
4.5	Componentes encontrados.	56
4.6	Componentes alterados e redimensionados.	56
4.7	Erro no decorrer das iterações do treinamento.	59
4.8	Centroides e limiares dos símbolos	61
4.9	Colisão de círculos.	62
4.10	MST gerada pelo sistema.	63
4.11	Exemplo de símbolos sobrepostos	66
4.12	Aplicação <i>web</i>	67
4.13	Ferramenta de seleção	67
4.14	Resultado do sistema	68
4.15	Inspecionar etapas do reconhecimento	68
5.1	Expressão básica	69
5.2	Denominador com subtração	69
5.3	Fração com expoentes	69
5.4	Integral dupla	70
5.5	Duplo expoente	70
5.6	Problema com o pré-processamento	70
5.7	Problema com o segmentação de símbolos justapostos	71
5.8	Problema com a raiz	71
5.9	Problema com o classificador	72
5.10	Problema com símbolos fora da região de dominância	72
5.11	Problema com centroides de símbolos em regiões erradas	73
5.12	Problema com a região de subexpressão	73

Lista de Quadros

3.1	Tabela comparativa da taxa de reconhecimento de símbolos para cada <i>feature</i> .	40
3.2	Tabela com as publicações, técnicas de extração de características e a <i>dataset</i> usada	41
3.3	Tabela comparativa dos classificadores	42
4.1	Arquitetura da CNN	58
4.2	Definição dos centroides	60
4.3	Definição dos limiares superior e inferior	60

1 Introdução

Escrever expressões matemáticas complexas em um computador usando editores de textos é uma tarefa trabalhosa. O teclado não possui todos os símbolos e, por isso, softwares como Microsoft Word e LibreOffice possuem modos de edição de fórmulas que disponibilizam tabelas de símbolos e operações para que o usuário possa montar uma expressão. A quantidade de símbolos utilizados na matemática é muito grande, chegando aproximadamente a 2 mil símbolos (BHARAMBE, 2015). Implementar uma interface gráfica que permita uma pessoa fazer uso de todos esses símbolos não seria viável. Uma ferramenta de busca por símbolos facilitaria, porém, em alguns casos, é difícil lembrar ou descrever um símbolo em um campo de busca.

Seria mais fácil se fosse possível escrever uma expressão matemática manualmente e em seguida o computador a identificasse e mostrasse na tela. Escrever uma expressão manualmente pode ser visto como desenhar a expressão através do mouse, usar o dedo em uma tela *touch*, usar uma caneta especial (*stylus pen*) para “desenhar” na tela, ou ainda, escrever a expressão em um papel com uma caneta. As pessoas estão acostumadas a escrever expressões matemáticas com papel e caneta, portanto, escrever uma expressão em um pedaço de papel para que o computador a reconhecesse, seria mais natural e apropriado.

Desenvolver um reconhecedor automatizado de expressões matemáticas, além de facilitar a entrada de expressões em um editor de textos, teria outras utilidades como a conversão de livros e artigos científicos para o formato digital, ajudar na entrada de dados em softwares especializados em manipulação algébrica, facilitar a criação de documentos LaTeX sem precisar conhecer os comandos, e até mesmo auxiliar portadores de deficiência visual pela leitura de documentos e papéis com expressões matemáticas (SIDER; OZAHATA; HIRATA, 2010).

Pelos diversos meios de utilização desse tipo de sistema, seria útil que o reconhecedor automatizado de expressões, após o reconhecimento, fornecesse a saída em um padrão que pudesse ser interpretado por vários softwares. Um padrão já consolidado é o MathML (AUS-BROOKS, 2003). Outra forma de representar fórmulas matemáticas é através de códigos LaTeX. Embora este não seja específico para isso, consegue não apenas expressar fórmulas complexas através de códigos, mas também renderizá-las com fontes apropriadas para visualização. Além disso, é possível importar códigos LaTeX para o Microsoft Word e LibreOffice através de

plugins. Por conta disso, este trabalho utiliza padrão de saída LaTeX.

O reconhecimento ótico de caracteres (sigla em inglês OCR) é uma área estudada há muitos anos (MARINAI; FUJISAWA, 2007) tanto para o reconhecimento de caracteres manuscritos quanto tipográficos. Essa área já está em um estágio muito avançado no reconhecimento de linguagens naturais (inglês, português, etc). Porém, quando se trata de reconhecimento de expressões matemáticas, ainda há muito o que ser feito (TAPIA, 2007). Um fator que dificulta o reconhecimento de expressões matemáticas é a disposição dos símbolos. Ao contrário das linguagens naturais onde o reconhecimento tende a ser linear (exceto na acentuação), os símbolos em uma expressão matemática podem aparecer em um arranjo bidimensional (SIDER; HIRATA, 2015). Perceba, pela fórmula (1.1), que as expressões podem crescer na horizontal e na vertical.

$$\sum_{i=1}^{10} \frac{\sqrt{i^2 + \frac{i}{2}}}{\pi} \quad (1.1)$$

Um outro fator que pode dificultar no desenvolvimento de um reconhecedor de expressões é que na matemática não existe um padrão de notação. Por exemplo, a representação de vetores:

$$u = (1, 5) \quad u = \langle 1, 5 \rangle \quad u = [1 \ 5] \quad u = \begin{bmatrix} 1 \\ 5 \end{bmatrix} \quad (1.2)$$

Todas são aceitas. As ciências exatas fazem muito o uso da matemática e cada uma acaba criando convenções para notações (SIDER; HIRATA, 2015). Por isso, nenhuma das formas de escritas estão erradas. Existem muitas notações matemáticas como a da álgebra, cálculo, matrizes, provas e teoremas, e muitas outras. Segundo Matsakis (1999), dificilmente um sistema reconhecedor de expressões conseguirá ser capaz de lidar com todas essas notações.

Uma expressão matemática pode ser reconhecida pelo software enquanto a pessoa escreve. Neste caso o reconhecimento é chamado *online*. Se a expressão já estiver escrita em um pedaço de papel ou em texto formatado, o reconhecimento é dito *offline*. As informações que serão reconhecidas serão diferentes, e por isso, ao projetar um reconhecedor de expressões é preciso decidir qual dos modelos de reconhecimento será implementado.

O reconhecimento de expressões matemáticas é dividido em três etapas. A primeira é a segmentação que tem como objetivo isolar cada símbolo da expressão. A segunda etapa é a do reconhecimento dos símbolos ou classificação e o objetivo é identificar cada um dos símbo-

los. A terceira etapa é a da construção de uma estrutura de dados (árvore) capaz de representar a expressão. As técnicas utilizadas em cada uma dessas etapas estão relacionadas com o tipo de reconhecimento *offline* ou *online*. O foco deste trabalho é implementar um reconhecedor *offline* de expressões matemáticas manuscritas.

1.1 Objetivo

O objetivo deste trabalho é desenvolver uma aplicação *web* em que o usuário possa fazer *upload* de uma imagem (que contém uma expressão matemática manuscrita) e o sistema deverá interpretar essa imagem e devolver para o usuário o código LaTeX da expressão, e ainda, o navegador deverá renderizá-la para que a pessoa possa visualizar.

1.2 Objetivos Específicos

Para alcançar a efetividade de um sistema de reconhecimento de expressões matemáticas manuscritas, é necessário garantir que as seguintes etapas sejam completadas:

- Estudar as técnicas utilizadas para segmentação, classificação e geração da árvore de expressões matemáticas.
- Definir uma base de dados de símbolos matemáticos para treinar o classificador.
- Implementar as etapas de segmentação e classificação.
- Treinar o classificador para reconhecer os símbolos isolados.
- Implementar a etapa da geração da árvore de expressões.
- Definir uma base de dados de expressões matemáticas para aplicar testes e garantir uma boa taxa de acertos.
- Aplicar testes para validar o sistema.
- Desenvolver uma aplicação *web* para que as pessoas possam usá-lo.

1.3 Metodologia

Este trabalho propõe a implementação de uma aplicação *web* para reconhecimento de expressões manuscritas, e por isso, possui natureza aplicada. São feitos estudos na literatura sobre os métodos utilizados em cada uma das etapas do reconhecimento de expressões. São mostrados resultados da aplicação, expressões que não foram reconhecidas corretamente e outras que tiveram sucesso no reconhecimento. Ao final é destacado os pontos fracos de cada método utilizado.

1.4 Estrutura do Trabalho

Este trabalho é dividido em cinco capítulos. O Capítulo 1 é uma introdução sobre o reconhecimento de expressões matemáticas, as possíveis aplicações, os tipos de reconhecimento e as etapas do reconhecimento. No Capítulo 2 são apresentados os fundamentos básicos de processamento de imagens, aprendizado de máquina e análise estrutural. O Capítulo 3 é uma revisão dos métodos utilizados na literatura para cada uma das etapas do reconhecimento de expressões matemáticas. O Capítulo 4 apresenta detalhes sobre a implementação da aplicação *web* apresentando quais métodos foram implementados em cada uma das etapas. O Capítulo 5 mostra os resultados obtidos após a implementação do sistema. O Capítulo 6 apresenta as considerações finais deste trabalho, o que foi estudado, o que foi implementado de maneira diferenciada dos outros autores, os pontos fracos dos métodos implementados e o que pode ser melhorado em trabalhos futuros.

2 Fundamentação Teórica

Neste capítulo são abordados conceitos importantes para o entendimento das técnicas que são utilizadas nos trabalhos relacionados (Capítulo 3) e na proposta do trabalho (Capítulo 4). São apresentadas definições básicas de processamento de imagens, métodos de segmentação, morfologia matemática, extração de características, aprendizado de máquina, algoritmos de classificação e análise estrutural de expressões matemáticas.

2.1 Processamento e reconhecimento de padrões em imagens

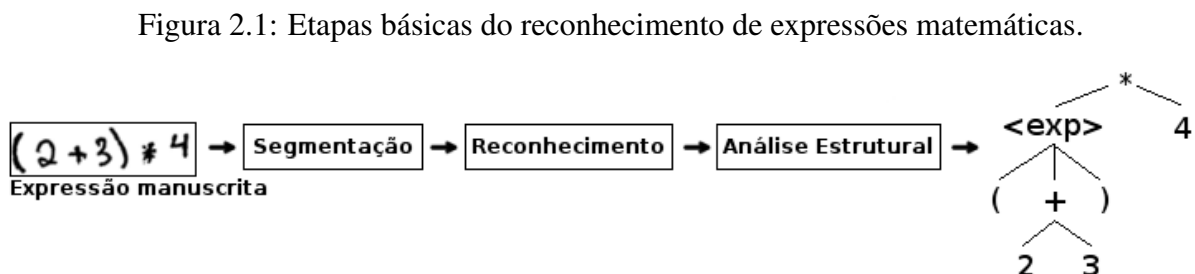
Processamento de imagens é uma área que estuda métodos para capturar, representar e transformar imagens com o auxílio do computador (GONZALEZ; WOODS, 2006). A primeira etapa, a da captura, estuda como é o processo de aquisição de uma imagem digital. As próximas etapas, dependem muito da aplicação que usará essa imagem. Para o desenvolvimento deste trabalho, é necessário citar os passos básicos para o reconhecimento de padrões em imagens. Após a **aquisição** da imagem, muitas vezes ela vem com artefatos (elementos não desejáveis), por isso, a próxima etapa é o **pré-processamento**, que prepara a imagem eliminando ruídos, removendo partes desnecessárias, mudando o sistema de cores e realçando regiões de interesse. A fase seguinte é a **segmentação**, que isola regiões de interesse na imagem. Após ter a imagem segmentada, o passo seguinte é **extrair características** que a descrevem. Essas informações extraídas são guardadas em um vetor que representará o conteúdo (NIXON, 2008). O próximo passo é classificar a imagem baseado nessas informações adquiridas. Esse passo da **classificação** pode ou não depender de um processo chamado **aprendizado de máquina**, onde é necessário usar uma base de dados para a comparação de informações de figuras que já foram reconhecidas. Primeiramente é preciso treinar o classificador e encontrar um modelo. Esse modelo pode ser entendido como uma função que separa as classes. No contexto desse trabalho cada símbolo (dígitos, letras, etc) da expressão matemática é uma classe. Após o treinamento, o classificador deverá classificar corretamente para as novas entradas. No final da etapa de classificação a imagem é reconhecida e agora possui um significado.

2.2 Digitalização

A digitalização é o processo de obter uma imagem digital a partir de uma cena. É feita em duas etapas: amostragem e quantização (GONZALEZ; WOODS, 2006). A **amostragem** é o processo de obter uma estrutura necessária para caber a imagem, ou seja, cria uma matriz com a resolução correta para a imagem adquirida. A **quantização** atribui valores a cada elemento dessa matriz, também chamados de *pixel*. Os valores de cada *pixel* representam a intensidade da cor. Para este trabalho, é utilizado apenas escalas de cinza, com a intensidade do pixel variando de 0 à 255 e imagens binárias com os valores 0 (preto) ou 1 (branco). Uma imagem também pode ser representada como uma função $f(x, y)$ onde o par (x, y) representam a posição do *pixel* na matriz.

2.3 Visão geral do reconhecimento de expressões matemáticas

O reconhecimento de expressões matemáticas é tradicionalmente dividido em três etapas básicas (MUÑOZ, 2015): segmentação dos símbolos, reconhecimento dos símbolos e análise estrutural. A segmentação se preocupa em separar cada símbolo da expressão a partir de uma imagem de entrada. O reconhecimento interpreta cada símbolo isoladamente. A análise estrutural constrói uma estrutura de dados (com os símbolos já identificados) de forma que represente uma expressão matemática obedecendo a ordem de precedência das operações. Veja a Figura 2.1.



Fonte: produzido pelo próprio autor.

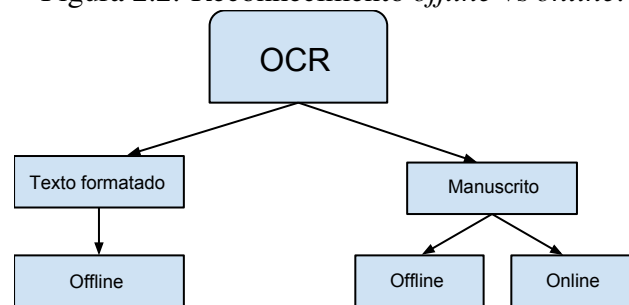
Existem outras etapas que podem ser adicionadas no processo mas nem todos os autores as consideram, como o pré-processamento antes da segmentação e a etapa de correção de erros depois da análise estrutural (TAPIA, 2005). Este trabalho considera apenas a etapa de

pré-processamento que será abordada a seguir.

2.4 Reconhecimento *online* vs *offline*

Os OCRs podem ser classificados em dois tipos: texto formatado (fontes de computador, máquinas de escrever, etc) ou texto escrito à mão. Para os textos manuscritos ainda é possível diferenciar se o texto está sendo analisado enquanto a pessoa escreve (*online recognition*) ou quando o texto já foi escrito (*offline recognition*).

Figura 2.2: Reconhecimento *offline* vs *online*.



Fonte: adaptado de Tapia (2005).

É possível realizar o reconhecimento *online* através de dispositivos como *stylus pen* (usada em *smartphones* e PDAs) *Interactive whiteboard* (lousa digital), desenhando na tela através do mouse ou usando o dedo em telas *touchscreen*. Usar reconhecimento *online* é vantajoso pois é possível coletar informações temporais da escrita armazenando pontos em um curto intervalo de tempo. Quando a pessoa começa a escrever, um evento *pen-down* é disparado e quando a caneta é levantada um evento *pen-up* é acionado (TAPIA, 2005). Com isso, é possível armazenar as sequências de traços (formados pelos pontos) na ordem que foi escrita. Assim, é possível resolver o problema de quando dois símbolos estão sobrepostos. Basta usar o histórico de traços para separá-los (TAPIA, 2005). Além das informações temporais é possível armazenar pressão, velocidade e aceleração.

Vários estudos já foram feitos e obtiveram bons resultados usando reconhecimento *online* (TAPIA, 2007), porém, esta técnica não resolve o problema de expressões que já foram escritas (em pedaços de papéis, por exemplo) ou impressas (em livros). Segundo Muñoz (2011), alguns estudos já foram feitos usando reconhecimento *offline*, mas ainda precisam melhorar muito, principalmente na análise estrutural. Por isso, este trabalho aborda o reconhecimento *offline* manuscrito.

2.5 Morfologia Matemática

A **morfologia matemática** é uma metodologia para análise e manipulação de estruturas geométricas em imagens digitais. Permite trabalhar com bordas, filtros, conectividade e outras técnicas de pré e pós-processamento (GONZALEZ; WOODS, 2006). No contexto do reconhecimento de expressões matemáticas as técnicas de morfologia são úteis para eliminar possíveis ruídos que podem aparecer na imagem. É fundamentada na teoria dos conjuntos utilizando-se das operações básicas: união, interseção, complemento, diferença e ainda acrescenta reflexão e a translação que são definidas da seguinte forma:

Seja A um conjunto pertencente a $\mathbb{Z}^2$, onde cada elemento é um par de coordenadas (x, y) que representam um pixel na cor preta ou branca. Segundo Pedrini e Schwartz (2008) a **reflexão** de A é:

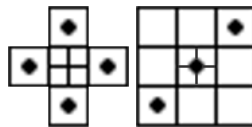
$$\hat{A} = \{-a | a \in A\} \quad (2.1)$$

Ou seja, a reflexão resulta em um conjunto em que cada elemento é a coordenada $(-x, -y)$. Já a translação de A por um ponto $z = (z_1, z_2)$, denotada por A_z , resulta em um conjunto cujos elementos são $(x + z_1, y + z_2)$. A **translação** é definida da seguinte forma (GONZALEZ; WOODS, 2006):

$$A_z = \{a + z | a \in A\} \quad (2.2)$$

O princípio básico da morfologia matemática é a utilização de um operador morfológico também chamado de **Elemento Estruturante** (EE), uma pequena imagem, que é transladada sobre a imagem de entrada aplicando operações e modificando a geometria da imagem (PEDRINI; SCHWARTZ, 2008). O EE binário é formado por pixels nas cores preto (1) e branco (0) e ainda possui uma origem normalmente ao centro como pode ser visto na Figura 2.3. As operações primitivas entre o EE e a imagem de entrada são **erosão** e **dilatação** que serão abordadas na sequência.

Figura 2.3: Imagem de dois EE com seus centros na origem.



Fonte: produzido pelo próprio autor.

2.5.1 Erosão e Dilatação

Sejam os conjuntos A e B pertencentes a $\mathbb{Z}^2$, a **erosão** de A pelo elemento estruturante B , também denotada por $A \ominus B$, é usada para a remoção de pequenos componentes ou ruídos na imagem. Segundo Gonzalez e Woods (2006) a erosão pode ser vista como uma filtragem morfológica. A remoção dos pequenos elementos acontecem quando o EE se encaixa totalmente na imagem A , marcando na imagem de saída apenas o pixel da mesma posição central do EE. A erosão é definida como:

$$A \ominus B = \{z | (B)_z \subseteq A\} \quad (2.3)$$

A equação (2.3) mostra a operação de translação do pixel central do EE sobre cada elemento da imagem A . O conjunto resultante serão todos os elementos z quando todos os pixels do EE são iguais aos do conjunto A .

Ao contrário da erosão, a dilatação aumenta os componentes da imagem. Sejam os conjuntos A e B pertencentes a $\mathbb{Z}^2$, a **dilatação**, também denotada por $A \oplus B$, resulta um conjunto com todos os elementos quando A intersecciona com B refletido. A dilatação é definida da seguinte forma (GONZALEZ; WOODS, 2006):

$$A \oplus B = \{z | (\hat{B})_z \cap A \neq \emptyset\} \quad (2.4)$$

As operações morfológicas erosão e dilatação possuem algumas propriedades onde ambas se relacionam. A operação de dilatação pode ser escrita como erosão usando a propriedade da dualidade (PEDRINI; SCHWARTZ, 2008).

$$[A \oplus B]^c = [A^c \ominus \hat{B}] \quad (dualidade) \quad (2.5)$$

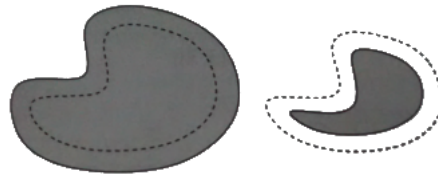
Com operações simples como complemento, reflexão e erosão é possível obter a dilatação. Essas operações exigem baixo custo computacional (DOUGHERTY; LOTUFO, 2003), por isso, é desejado que a operação de dilatação seja definida como:

$$A \oplus B = [A^c \ominus \hat{B}]^c \quad (2.6)$$

A operação de dilatação move o EE B sobre a imagem A e toda vez que o centro do EE encontrar um pixel marcado na imagem A , a imagem resultante terá todos os elementos do EE a partir do seu centro.

Na Figura 2.4 a imagem original é a pontilhada. O EE é um pequeno círculo. Como pode ser visto a dilatação ampliou todos os cantos da imagem e a erosão diminui toda a imagem

Figura 2.4: Dilatação à esquerda e Erosão à direita.



Fonte: (PEDRINI; SCHWARTZ, 2008).

de maneira uniforme. Isso só ocorreu devido ao EE ser um pequeno círculo com o elemento central no meio do círculo.

2.5.2 Abertura e Fechamento

A operação morfológica de **abertura** de um conjunto A por um EE B , é definida como a erosão de A por B seguida de uma dilatação por B (PEDRINI; SCHWARTZ, 2008).

$$A \circ B = (A \ominus B) \oplus B \quad (2.7)$$

A abertura é utilizada para eliminar conexões estreitas, remover saliências nas extremidades e suavizar o contorno dos objetos.

O **fechamento** de um conjunto A por um EE B , denotado por $A \bullet B$, é definido como:

$$A \bullet B = (A \oplus B) \ominus B \quad (2.8)$$

Como pode ser visto, o fechamento muda a ordem das operações em relação à abertura. Primeiro aplica-se a dilatação e em seguida uma erosão. O fechamento é usado para preencher pequenas lacunas nas extremidades, junta as discontinuidades estreitas da imagem e, também como a abertura, suaviza os contornos dos objetos.

As duas operações juntas são usadas combinadas em sequência para construir filtros que eliminam ruídos na imagem. Para eliminar pequenos ruídos aleatórios basta aplicar uma abertura e em seguida um fechamento ambas operações utilizando o mesmo EE. Essa técnica é frequentemente utilizada na etapa de pré-processamento.

2.6 Pré-processamento da imagem

O processo de digitalização de uma imagem pode trazer muitos ruídos em uma imagem (GONZALEZ; WOODS, 2006). Existem vários tipos de ruídos: uniforme, gaussiano, sal-

e-pimenta, entre outros. O pré-processamento da imagem é muito importante pois elimina esses artefatos da imagem.

2.6.1 Filtragem

As filtrações são operações que podem ser aplicadas a uma imagem com o objetivo de diminuir a quantidade de ruídos, suavizar, borrar, realçar regiões de interesse como bordas, entre outros. Normalmente as filtrações são feitas no **domínio espacial** da imagem que é o próprio espaço bidimensional e são chamados de filtros espaciais. Porém, os filtros também podem ser aplicados no **domínio da frequência** (GONZALEZ; WOODS, 2006). Para isso, pode-se utilizar a transformada discreta de Fourier (DFT) para transformar para este domínio. Em seguida, a imagem é modificada pelo filtro e depois é aplicado a transformada inversa para que a imagem volte ao domínio espacial.

Os filtros espaciais consistem em uma máscara ou *kernel* (pequena matriz) onde seus elementos são multiplicados pelos valores de intensidade da imagem original. Depois é feita soma de todas as multiplicações e o resultado é atribuído à uma nova imagem na posição do pixel central da máscara. A máscara é aplicada por toda a imagem original. A aplicação do filtro resultará em uma nova imagem com as mesmas dimensões. Gonzalez e Woods (2006) define a filtração linear espacial de uma imagem de dimensão $M \times N$ com uma máscara de tamanho $m \times n$ como:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (2.9)$$

onde $a = \frac{m-1}{2}$, $b = \frac{n-1}{2}$ e x, y devem ser variados até que cada pixel de w visite todos os pixels de f .

O mecanismo de filtração explicado até o momento chama-se **correlação**. A outra forma de filtração é chamada de **convolução** e a única diferença é que antes de aplicar o filtro de convolução, a máscara é rotacionada em 180° . Se a máscara for simétrica, os resultados da convolução e da correlação são os mesmos (GONZALEZ; WOODS, 2006). Na literatura, o termo convolução é mais utilizado, e se refere de forma geral, a filtração espacial.

As máscaras, algumas vezes chamadas de máscaras de convolução ou *kernels* são preenchidas com coeficientes, e estes, definem o tipo da filtração. Alguns tipos de máscaras são:

- Passa Baixa (suavização)
- Passa Alta (realce)
- Filtro de mediana e média (redução de ruídos)
- Passa banda
- Gradientes: Sobel e Robert (detectores de borda)

2.6.2 Limiarização

A **limiarização**, também chamada de binarização, é uma técnica de segmentação que consiste na classificação dos pixels conforme um ou mais limiares (PEDRINI; SCHWARTZ, 2008). É frequentemente utilizada quando deseja-se separar o fundo da imagem de um ou mais objetos que estão na cena. Seja uma imagem $f(x, y)$ em tons de cinza, é possível fazer essa separação do fundo dos objetos definindo um limiar T que separe esses dois grupos. Para cada ponto (x, y) da imagem f , se $f(x, y) > T$, esse ponto é considerado pertencente ao objeto, caso contrário, será um ponto do fundo da imagem. Formalizando este conceito, uma imagem limiarizada $g(x, y)$, é determinada como:

$$g(x, y) = \begin{cases} 0 & \text{se } f(x, y) \leq T \\ 1 & \text{se } f(x, y) > T \end{cases} \quad (2.10)$$

Desta forma, a imagem de saída será uma imagem binária com o objeto destacado de preto e o fundo destacado de branco, ou vice-versa. Quando esse processo utiliza apenas um limiar, é chamado de limiarização global. Porém, quando existem muitos objetos com variações no nível de cinza por conta da iluminação, esse processo não é adequado. Para esse caso, existe a limiarização múltipla, onde é definido vários limiares. Ainda existe a limiarização local, onde o processo é aplicado em pequenas regiões. Para o reconhecimento de expressões matemáticas manuscritas, é considerado que a expressão inteira é escrita da mesma cor, por isso, é utilizado apenas limiarização global.

2.7 Segmentação

Segmentação é o processo de subdividir ou isolar uma região de interesse em uma imagem. O processo de segmentação deve parar assim que alcançar um objetivo (GONZALEZ;

WOODS, 2006). No contexto da análise de expressões matemáticas, o objetivo é isolar cada símbolo da expressão matemática. A Figura 2.5 mostra à esquerda a imagem original (a), e a direita, os símbolos segmentados (b). Note que nesta etapa os símbolos ainda não são reconhecidos, são apenas imagens de interesse que em uma próxima etapa deverão ser identificados.

Figura 2.5: Expressão antes e depois de segmentada



Fonte: produzido pelo próprio autor.

A segmentação é a fase mais difícil do reconhecimento *offline* de expressões matemáticas. As subseções seguintes abordam vários fundamentos utilizados durante o processo da segmentação.

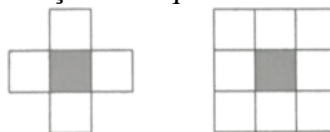
2.7.1 Vizinhança

A vizinhança de um pixel f é um conjunto de pixels que se encontram em volta deste (PEDRINI; SCHWARTZ, 2008). A **vizinhança-4**, denotada por $N_4(f)$, possui como pixels vizinhos os da horizontal e os da vertical. Para um pixel com coordenadas (x, y) , a vizinhança-4 deste pixel são os pixels das seguintes coordenadas: $(x+1, y)$, $(x-1, y)$, $(x, y+1)$ e $(x, y-1)$. A vizinhança das diagonais do pixel f , chamada de $N_d(f)$, possui as seguintes coordenadas: $(x-1, y-1)$, $(x-1, y+1)$, $(x+1, y-1)$ e $(x+1, y+1)$. A **vizinhança-8** de f , denotada por $N_8(f)$, possui a mesma vizinhança de $N_4(f)$ e mais as vizinhanças das diagonais.

$$N_8(f) = N_4(f) \cup N_d(f) \quad (2.11)$$

Na Figura 2.6 pode ser observado as regiões vizinhas de um pixel.

Figura 2.6: Vizinhança-4 à esquerda e vizinhança-8 à direita.



Fonte: (PEDRINI; SCHWARTZ, 2008).

2.7.2 Rotulação de Componentes Conexos

Rotulação (*labeling*) é o processo de identificar e atribuir números aos diversos componentes existentes em uma imagem (CHAN; YEUNG, 2000). No contexto do reconhecimento de expressões matemáticas, cada símbolo da expressão é um componente.

Considerando uma imagem binária onde o fundo é branco e o objeto é preto, a técnica de rotulação percorre a imagem e agrupa seus componentes baseado na conectividade dos pixels. A conectividade depende de qual tipo de vizinhança foi escolhida (vizinhança-4 ou 8). A imagem é percorrida pixel por pixel (do topo da esquerda para a direita) até encontrar um pixel preto, esse pixel então é marcado como um novo componente. Na sequência é feita uma busca por todos os pixels adjacentes (vizinhos) e para cada um deles é feita a mesma marcação (número) do primeiro pixel daquele componente. Depois, para cada pixel vizinho encontrado é feito o mesmo processo recursivamente até não encontrar mais vizinhos. A imagem é varrida atribuindo uma nova marcação para cada novo componente encontrado até terminar a imagem.

2.8 Extração de Características

Na área de reconhecimento de padrões, extrair características de uma imagem pode ser considerado uma forma de redução de dimensionalidade, ou seja, selecionar apenas informações relevantes para diminuir o número de dados que serão analisados (NIXON, 2008). As informações extraídas são guardadas em um **vetor de características**.

Um exemplo simples seria transformar uma imagem em um vetor onde os elementos são os valores dos tons de cinza de cada pixel da imagem. Cada elemento é um número que varia de 0 a 255 onde zero é preto, 255 é branco e os valores intermediários são os tons de cinza. Essa é uma forma básica de obter uma imagem digital, porém, a quantidade de informação que esse vetor guarda, dependendo da resolução da imagem, pode ser muito grande. Além disso, esse vetor guarda informações muito primitivas (pontos). Por isso, esse vetor têm **características de baixo nível** (*low-level features*) (NIXON, 2008).

Extrair informações relevantes de imagens significa descrever a imagem com **características de alto nível** (*high-level features*). Conseguir extrair informações como a cor da pele de uma pessoa, a cor do olho, ou ainda identificar se na imagem existe um cachorro, são informações que descrevem melhor a cena. Encontrar esse tipo de descrição reduz bastante o

número de informações que precisam ser analisadas e isso ajuda muito na próxima etapa do reconhecimento que é o aprendizado de máquina.

As técnicas utilizadas para descrever uma imagem não são genéricas, ou seja, dependem muito do que está querendo ser analisado. Na seção dos trabalhos relacionados são abordadas técnicas utilizadas para obter descritores de imagens contendo expressões matemáticas.

2.9 Aprendizado de Máquina

A etapa do reconhecimento de expressões matemáticas é responsável por atribuir significado aos símbolos da expressão. Para isso, é necessário utilizar algum conhecimento prévio de cada um dos símbolos. A área de reconhecimento de padrões em imagens se resume em atribuir rótulos à objetos (KUNCHEVA, 2004). Esse processo determina um mapeamento entre as propriedades extraídas da imagem com um conjunto de rótulos (PEDRINI; SCHWARTZ, 2008). Para fazer esse mapeamento é necessário utilizar técnicas capazes de analisar um conjunto de dados e tirar conclusões. **Aprendizado de Máquina** (AM) é a área que estuda métodos capazes de detectar padrões e fazer previsões sobre dados (MURPHY, 2012). Em vez de criar um programa que resolve uma tarefa, com AM é possível criar um programa capaz de aprender a como resolver uma tarefa. Isso abre um leque de possibilidades que reduz a intervenção humana na resolução de problemas.

Autores geralmente dividem AM em dois grupos: aprendizagem supervisionada e não-supervisionada. A **aprendizagem supervisionada** ou **preditiva** analisa dados já conhecidos (rotulados) e gera um modelo capaz de prever, para novos dados, quais classes (rótulos) ou valores que os caracterizam (MURPHY, 2012). Já a **aprendizagem não-supervisionada** ou **descritiva** é usada quando não se tem conhecimento a respeito do conjunto de dados e deseja-se encontrar propriedades que descrevam esse conjunto. As amostras fornecidas ao sistema são analisadas e o algoritmo associa os dados mais parecidos, formando um **agrupamento** (*clustering*).

No aprendizado supervisionado, quando o problema é decidir qual classe (rótulo) um dado pertence, o problema é chamado de **classificação**. Quando o objetivo é prever um valor real (contínuo), o problema é chamado de **regressão** (MURPHY, 2012).

A Figura 2.7 mostra de forma simples a divisão que os autores fazem a respeito

Figura 2.7: Modelos de Aprendizado de Máquina.



Fonte: Produzido pelo autor.

dos problemas de AM. A hierarquia se estende ainda mais no aprendizado não-supervisionado, porém, não é do escopo deste trabalho, portanto, não é abordado.

Nas próximas subseções são abordados vários algoritmos de classificação que são de interesse da área de reconhecimento de expressões.

2.9.1 K-Vizinhos mais próximos

O método dos **k-vizinhos mais próximos**, em inglês *k-Nearest Neighbour* (k-NN), é uma das técnicas de AM mais simples. Pode ser usado para problemas de classificação e regressão. Para este trabalho é considerado apenas classificação. Este método utiliza um conjunto de treinamento, isto é, dados já conhecidos (rotulados). Quando um novo dado precisa ser classificado, o algoritmo compara-o com os dados de treino e descobre quais são os dados mais parecidos, e assim, assume-se que o novo dado pertence à mesma classe (PEDRINI; SCHWARTZ, 2008).

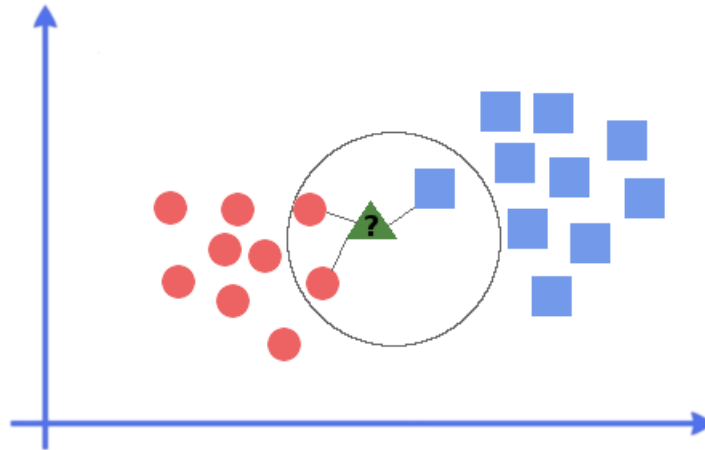
Cada dado pode ser representado por um ponto no espaço e sua localização é determinada pelos seus atributos. Existem várias métricas que calculam o quão parecidos são dois objetos. A mais comum é a Distância Euclidiana que calcula a distância em linha reta entre dois pontos. Seja **M** o número de atributos de um objeto **s** a ser identificado, e **k** um objeto conhecido, segundo Nixon (2008) a Distância Euclidiana é:

$$d = \sqrt{\sum_{i=1}^M (s_i - k_i)^2} \quad (2.12)$$

Para decidir à qual classe um objeto pertence, é definido um valor **k** que é o número de objetos vizinhos mais próximos (parecidos). Na Figura 2.8, existem duas classes de objetos.

Cada objeto possui atributos que podem ser formato, tamanho, cor etc. Deseja-se descobrir qual classe pertence o novo objeto (marcado com ponto de interrogação). Para classificá-lo, o método k-NN verifica quais são os k objetos mais próximos, que neste caso, são três ($k=3$). Como possuem apenas duas classes, foi escolhido um número k ímpar justamente para desempatar. Neste caso, o objeto é classificado na classe dos círculos porque dois dos três vizinhos mais próximos eram círculos.

Figura 2.8: k-NN com $k=3$.



Fonte: Produzido pelo autor.

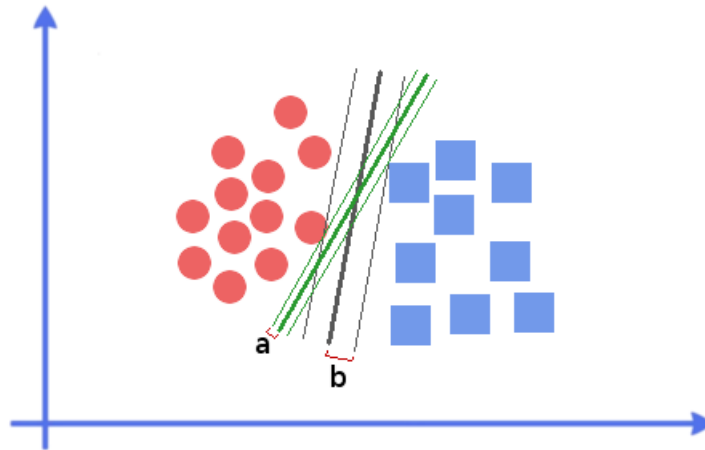
2.9.2 Máquina de vetores de suporte

Máquina de vetores de suporte ou *Support Vector Machine* (SVM) é outro método de AM supervisionado que pode ser utilizado para classificação e regressão. O SVM classifica um objeto de entrada em uma de duas possíveis classes. Por isso, o SVM é um classificador binário, já o k-NN, pode ser usado para um número arbitrário de classes.

O objetivo do SVM é encontrar o melhor hiperplano que separa os dados em duas classes (VAPNIK; VAPNIK, 1998). É chamado de hiperplano porque os dados podem conter uma quantidade qualquer de atributos. O melhor hiperplano é aquele que possui uma maior margem de separação dos dados, ou seja, aquele que se ajusta para que fique simultaneamente o mais distante possível dos dados que estão mais próximos do hiperplano.

Na Figura 2.9, os valores a e b são as margens de separação. O melhor hiperplano é aquele que tem a maior margem, neste caso, $b > a$, portanto, esse hiperplano consegue separar melhor os dados. O problema de encontrar o melhor hiperplano é um problema de otimização que pode ser resolvido com os multiplicadores de Lagrange (VAPNIK; VAPNIK, 1998).

Figura 2.9: Classificador SVM encontrando o melhor hiperplano.



Fonte: Produzido pelo autor.

Como visto anteriormente, o SVM classifica os dados por meio de um hiperplano, e dessa forma, é um classificador linear. Porém, é possível classificar problemas não lineares utilizando uma técnica que aplica uma função de transformação no conjunto de dados afim de aumentar a dimensão do problema. Isso ajuda a separar dados que antes ficavam com pouca ou até mesmo sem margem do hiperplano (VAPNIK; VAPNIK, 1998).

2.9.3 Redes Neurais Artificiais

As **Redes Neurais Artificiais** (RNAs) são técnicas de classificação muito populares. São modelos matemáticos inspirados no funcionamento do sistema nervoso biológico (TAPIA, 2005). As RNAs são altamente flexíveis, ou seja, não estão restritas à problemas específicos e são capazes de representar funções com alto grau de complexidade, e por isso são muito utilizadas (PEDRINI; SCHWARTZ, 2008).

Podem ser vistas como um grafo orientado em que os nós são os neurônios e as arestas representam as conexões de entradas e saídas entre os neurônios artificiais (PEDRINI; SCHWARTZ, 2008). O primeiro modelo foi proposto por McCulloch e Pitts (1943) onde o neurônio calcula um limiar binário, ou seja, uma função em que a saída tem dois possíveis valores: -1 e $+1$. A entrada é representada por um vetor $X = [x_1, x_2, \dots, x_n]$ com n elementos. No contexto deste trabalho a entrada é o vetor de características extraídas do símbolo já segmentado. Cada uma das entradas possui um peso associado que também pode ser representado por um vetor $W = [w_1, w_2, \dots, w_n]$ possuindo n elementos. É feita a soma de cada entrada multiplicada pelo peso de mesmo índice, ou seja, a soma ponderada pelos pesos. Segundo Pedrini e Schwartz

(2008) pode-se definir essa soma ponderada como:

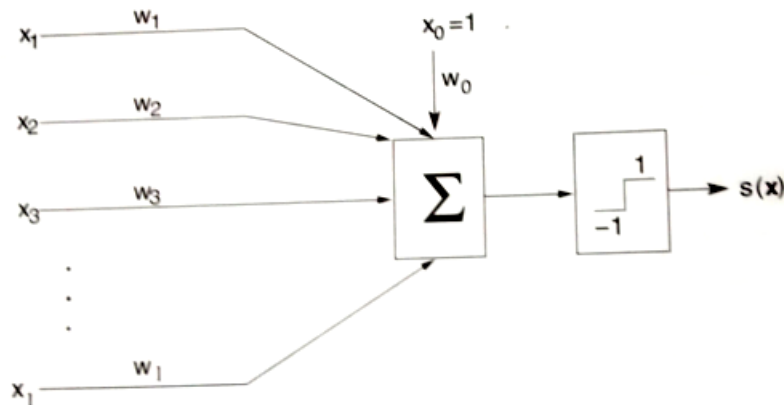
$$p = \sum_{i=1}^n w_i x_i \quad (2.13)$$

Depois o resultado é aplicado à uma **função de ativação** que pode ser interpretada como a informação de saída de um neurônio. Nesse modelo mais simples, também chamado de **Perceptron**, a função de ativação, apresentada por McCulloch e Pitts (1943), pode ser definida como:

$$s(p) = \begin{cases} +1 & p > 0 \\ -1 & \text{caso contrário} \end{cases} \quad (2.14)$$

Esse modelo tem um problema quando o valor de todos os pesos são iguais a zero. Quando isso acontece, as entradas acabam sendo anuladas (GONZALEZ; WOODS, 2006). Para resolver esse problema, é acrescentado o valor 1 (comumente chamado de *bias*), forçando a ativação da função. Na Figura 2.10 pode ser visto o modelo Perceptron com esse acréscimo representado pelo x_0 :

Figura 2.10: Modelo de RNA com um neurônio Perceptron.



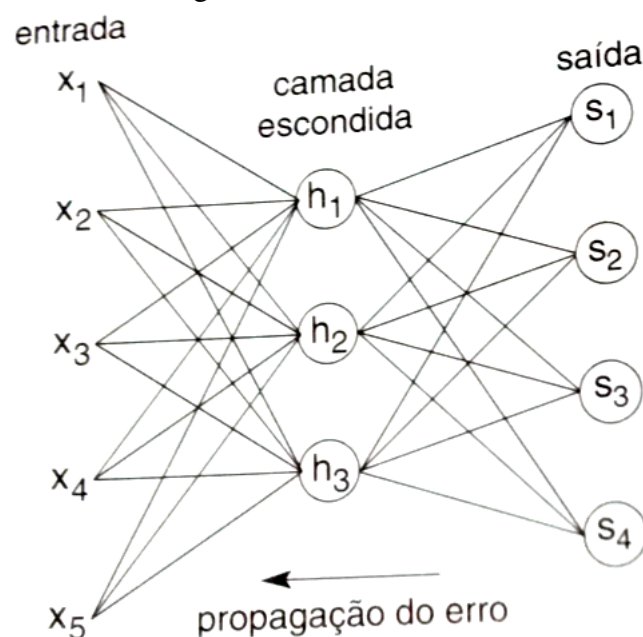
Fonte: (PEDRINI; SCHWARTZ, 2008).

A evolução desse modelo é o **MultiLayer Perceptron** (MLP). Esse modelo possui três tipos de neurônios: de **entrada**, de **saída** e os neurônios **escondidos** ou **ocultos**. A rede pode conter vários neurônios de entrada, vários níveis (camadas) de neurônios escondidos e vários neurônios de saída. A informação flui na rede deslocando em um único sentido até chegar nos neurônios de saída. Esse tipo de rede é conhecida como *feedforward neural network* (GONZALEZ; WOODS, 2006). Existem outros tipos onde neurônios podem voltar nas camadas anteriores da rede formando ciclos. Esse tipo é conhecido como Rede Neural Recorrente.

A ideia por trás das RNAs é apresentar um grande conjunto de exemplos para que a rede possa **treinar/aprender**. Esse aprendizado acontece porque os dados apresentados já estão

classificados, e por isso, a rede pode saber se o resultado que ela está informando está correto ou não. Esse processo de treinamento pode ser implementado usando uma técnica chamada **backpropagation** que é dividida em duas etapas (GONZALEZ; WOODS, 2006). A primeira é conhecida por *forward* onde a informação flui até os nós de saída. Depois é feito o cálculo do erro para que seja possível reajustar os pesos da rede. Esse cálculo é feito comparando os valores obtidos com os valores desejados. Após obter o erro é necessário fazer o caminho inverso, corrigindo os pesos a partir dos nós de saída até os primeiros nós escondidos. Essa etapa é chamada de *backward*. O treinamento é feito várias vezes. Depois de feito o treinamento, deseja saber se a RNA consegue classificar uma entrada qualquer. Se a nova entrada for parecida, a RNA tem mais chance de acertar. A Figura 2.11 mostra uma rede MLP com o sentido de correção dos pesos.

Figura 2.11: Modelo MLP.



Fonte: (PEDRINI; SCHWARTZ, 2008).

Modelos mais recentes fazem uso de funções de ativação mais avançadas que admitem mais que dois possíveis valores de saída. Ainda é possível criar modelos **heterogêneos** de RNAs, em que neurônios diferentes podem ter funções de ativação diferentes. Porém, é comum implementar o modelo **homogêneo** usando a mesma função para todos os neurônios.

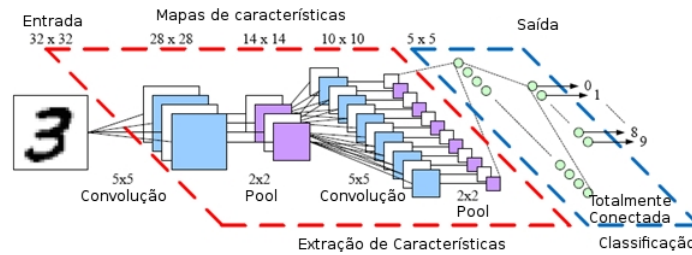
As Redes Neurais Convolucionais (*Convolutional Neural Network*, CNN ou *conv-net*) são RNAs modificadas especificamente para operar sobre imagens. Baseada no conceito de convolução, as CNNs compõem suas camadas de neurônios por matrizes de convolução aplicadas na imagem original. Cada operação de convolução indica a aplicação de um filtro ou *kernel*

na imagem de entrada. A primeira camada da CNN (*input layer*) é a própria imagem. No contexto do reconhecimento de expressões, a entrada é a imagem de um símbolo segmentado. A primeira camada intermediária (*hidden layer*) é uma matriz de neurônios chamada de *feature map* (NIELSEN, 2015). Os pesos das arestas são os valores da matriz de convolução. Cada neurônio dessa matriz guarda o resultado da convolução em apenas uma parte da imagem. Essa parte da imagem é chamada de *Local Receptive Fields*. São utilizados vários tipos de filtros, cada um será responsável por ativar certas informações da imagem e juntos irão conseguir extrair informações relevantes da imagem de entrada. Portanto, para cada filtro utilizado, existirá mais uma camada na CNN. Além das camadas das convoluções, uma outra camada chamada *pooling* é utilizada para cada um dos *feature maps* com o objetivo de reduzir a quantidade de informação ativando apenas as regiões mais importantes das camadas das convoluções (NIELSEN, 2015). A última camada é reduzida as classes desejadas do problema, neste caso, os símbolos matemáticos presentes no conjunto de dados. O treinamento de uma CNN funciona da mesma forma que uma MLP com *feedforward* e *backpropagation*.

Normalmente o reconhecimento de padrões requer que o projetista escolha quais são os principais descritores que aplicados na imagem irão extrair informações relevantes que ajudarão o classificador decidir sobre as classes. Porém, classificadores podem ser projetados para extrair características da imagem de entrada de forma automática. Esse método é chamado extração implícita de características (FERNANDES, 2013).

As CNNs possuem extração implícita de características através dos filtros aplicados na imagem de entrada (FERNANDES, 2013). Esses filtros de convolução (*kernels*) são representados por matrizes. Os coeficientes das matrizes são os pesos dos neurônios que serão otimizados. Portanto, o processo de treinamento da CNN tem como objetivo descobrir os coeficientes das matrizes de convolução (filtros) que melhor separam as classes da *dataset*. A Figura 2.12 mostra uma região tracejada em vermelho representando o conjunto de camadas responsáveis pela extração de características e outra região em azul, ao final da rede, que representa camadas de neurônios totalmente conectadas, similar aos neurônios de uma MLP. Entre cada convolução, pode-se observar a operação de *pooling* sendo aplicada sempre reduzindo o tamanho dos mapas de características pela metade.

Figura 2.12: Arquitetura de uma CNN.



Fonte: adaptado de Ward (2011).

2.10 Análise estrutural

A etapa do reconhecimento (classificação) dos símbolos resulta em um vetor contendo os símbolos já rotulados. Porém, esses símbolos não possuem nenhum relacionamento entre si. A próxima etapa é construir uma estrutura hierárquica que pode ser representada por uma **árvore de derivação** (*parse tree*) (CHAN; YEUNG, 2000) para organizar todos os termos, operadores e relacionamentos que constituem a expressão matemática. No entanto, nem todos os operadores podem aparecer explicitamente na expressão, fazendo com que o vetor resultante da etapa do reconhecimento, não contenha todos os símbolos. Veja as seguintes expressões:

$$x^2 \quad \text{e} \quad 5x$$

Note que não existe a presença explícita de um operador de potência. No caso da multiplicação, apenas o fato de dois símbolos estarem próximos já representam a multiplicação. No primeiro caso, a posição e o tamanho do símbolo (expoente) são atributos importantes para compreender a operação de potenciação. Já no segundo caso, apenas a posição. Portanto, durante o processo de construção de uma árvore de derivação, é necessário encontrar esses operadores que nem sempre são visíveis.

2.10.1 Relação espacial e atributos dos símbolos

Os símbolos do alfabeto matemático possuem vários tipos de grafias e isso dificulta o processo de encontrar o centroide e os limites superior e inferior de um símbolo (SIDER; OZAHATA; HIRATA, 2010). Por isso, é necessário classificar como os símbolos podem estar distribuídos e quais são seus atributos.

- **Centroide:** Segundo Zanibbi, Blostein e Cordy (2002) o centroide é um ponto usado

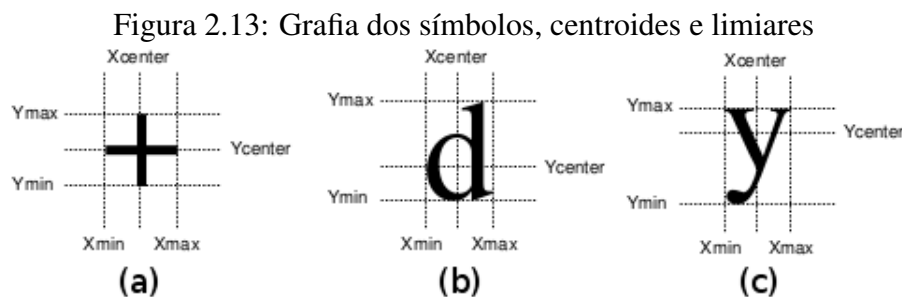
para testar se um símbolo encontra-se dentro de uma região da imagem. Substituir um símbolo inteiro por um único ponto facilita a análise geométrica. Esta é uma técnica comum na literatura e foi introduzida primeiramente por Anderson (1967). A localização do centroide no eixo x é sempre na metade do *bounding box*: $\frac{\min X + \max X}{2}$. Já a posição do eixo y varia conforme o tipo de grafia do símbolo: ascendente, descendente ou central.

- **Limiares verticais:** atributos numéricos que representam as limitações do eixo vertical do símbolo. Um símbolo y está na região superior do símbolo x se o limiar inferior de y está acima do centroide de x .
- **Limiares horizontais:** atributos numéricos que representam as limitações do eixo horizontal do símbolo. Um símbolo y está na região da direita de x se o limiar da esquerda de y for maior que a do centroide de x .

A grafia dos símbolos pode ser classificada em três grupos:

- **Regulares ou Centrais:** $+$, $-$, a , c , e , j , m , n , o , r , s , u , v , w , x , z , $\sum$, $\sqrt{\quad}$, (e)
- **Ascendentes:** $0..9$, b , d , f , h , i , k , l , t , $A..Z$
- **Descendentes:** g , p , q , y

Na Figura 2.13 pode ser observado os limiares verticais e horizontais dos símbolos e também quanto a classificação da grafia: (a) regular ou central; (b) ascendente; (c) descendente. Observe a localização dos centroides. Embora a imagem possua símbolos tipográficos, as propriedades são mantidas para símbolos manuscritos.



Fonte: adaptado de (CHAN; YEUNG, 2000).

Segundo Sider e Hirata (2015) é possível distinguir as relações espaciais entre os símbolos (esquerda, direita, acima, abaixo, acima e à esquerda, acima e à direita, sub-expressão,

etc) de acordo com a posição do centroide em relação ao retângulo que abriga o símbolo também chamado de *bounding box*. Esse retângulo é obtido na etapa de segmentação quando os símbolos são rotulados.

2.10.2 Aspecto Comportamental

Cada símbolo possui relacionamentos espaciais distintos com outros símbolos. Por isso, é necessário dividir os símbolos em categorias de acordo com as características específicas do símbolo (SIDER; OZAHATA; HIRATA, 2010).

O operador de **raiz**, por exemplo, pode ser classificado em uma única categoria. A sua região superior esquerda possui um significado específico (radicando) e a parte interna do símbolo contém uma sub-expressão. Dessa forma, esse símbolo deve possuir obrigatoriamente uma sub-expressão e um possível radicando, e por isso, além das características geométricas, estas propriedades tornam este símbolo notável (SIDER; OZAHATA; HIRATA, 2010).

Outro símbolo que merece ser classificado em uma única categoria é a **barra horizontal**. Dependendo da localização espacial dos argumentos da barra, este pode assumir operadores com significados diferentes. A presença simultânea de símbolos nas regiões superior e inferior à barra, podem representar o operador de divisão. Já na ausência simultânea de símbolos nas regiões superior e inferior à barra e a presença de símbolos nas regiões esquerda e direita da barra, podem denotar o operador de subtração. A decisão de qual símbolo se trata, geralmente é associada, além dos aspectos comportamentais, aspectos geométricos. Por exemplo, a coordenada y do centroide da barra também pode ser uma informação útil para determinar qual operador se trata.

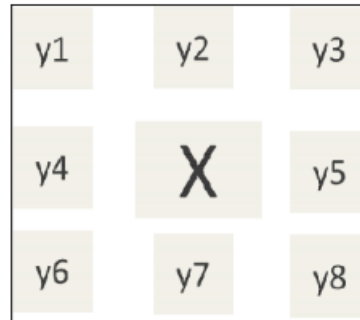
É possível definir muitas categorias de acordo com o comportamento dos símbolos em relação aos símbolos próximos. Segundo Sider, Ozahata e Hirata (2010) é possível definir para cada símbolo uma categoria. Porém, essa prática gera uma quantidade muito grande de categorias. Por isso, é importante definir um escopo de quais símbolos irão ser reconhecidos e quais são suas **classes comportamentais**.

2.10.3 Dominância

Uma forma de agrupar os símbolos de forma hierárquica é usando o conceito de dominância. A dominância ajuda a formalizar as classes comportamentais dos símbolos através

da posição e do tamanho relativo dos símbolos. O **alcance** ou **abrangência** de um símbolo é toda a área que situam seus **argumentos** ou operandos (TAPIA, 2005).

Figura 2.14: Possíveis regiões de alcance de um símbolo.

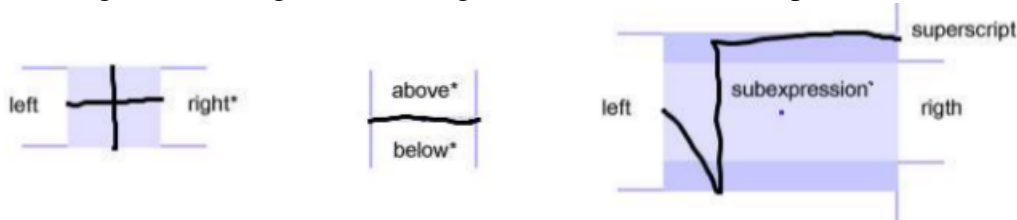


Fonte: (JAKJOURD; LAZREK, 2011).

Tapia (2005) define dominância da seguinte forma: um símbolo s **domina** um símbolo a se a está na região de alcance de s e s não está na região de alcance de a . A dominância pode ser definida como uma função booleana $domina(s, a)$ que depende de um conjunto de classes comportamentais $C = \{-, \sqrt{}, etc\}$.

Note que nem todos os símbolos possuem a mesma região de alcance. A Figura 2.14 mostra todos os possíveis alcances de um símbolo. Já a Figura 2.15 mostra o alcance de algumas classes comportamentais. Essa distinção é importante para evitar que dois símbolos próximos estejam ambos em suas regiões de abrangência. Se isso ocorrer, estratégias podem ser usadas para eliminar esse impasse, por exemplo, tomar que o símbolo com maior tamanho domina o símbolo de menor tamanho, ou ainda, considerar que o símbolo da esquerda domina o símbolo da direita, entre outras.

Figura 2.15: Regiões de abrangência de três classes comportamentais.



Fonte: adaptado de (TAPIA, 2005).

A Figura 2.15 mostra as regiões as quais estes símbolos podem dominar outros. O símbolo $+$ possui duas regiões: esquerda e direita. Observe que a direita possui um asterisco mostrando a obrigatoriedade de existir outro símbolo à direita. Se existir símbolos nos dois lados este símbolo representa a soma, senão, este símbolo enfatiza o sinal positivo na frente de

um número. A barra horizontal para ser considerada como símbolo da divisão precisa existir obrigatoriamente símbolos acima e abaixo dela. Já o símbolo da raiz, o radicando é opcional, mas a parte interna (sub-expressão) deve existir.

Como citado anteriormente, a dominância também considera o tamanho dos símbolos. Dois exemplos clássicos são a **subscrita** (*subscript*) e a **sobrescrita** (*superscript*). Por exemplo, em x^2 , x domina 2 por tamanho e em x_i , x domina i também por tamanho e, em ambos os casos, por pertencerem a região de alcance de x .

Esses conceitos fundamentais da análise estrutural são utilizados pela maioria dos autores. Porém, podem apresentar variações de acordo com o modelo de reconhecimento *offline* ou *online*. No capítulo dos trabalhos relacionados serão abordadas técnicas para construção de árvores de derivação com base nos conceitos apresentados nesta seção.

2.11 Grafos

Gersting (2004) define um grafo como uma tripla ordenada (N, A, g) onde N representa um conjunto não-vazio de vértices (nós), A representa um conjunto de arestas (arcos) e g uma função que associa cada aresta de A a um par não ordenado de vértices pertencentes a N . Um grafo é dito completo quando cada vértice possui arestas para todos os outros vértices do grafo.

No contexto do reconhecimento de expressões matemáticas, o uso de grafos é útil para a construção da árvore que representa a expressão matemática. Cada símbolo da expressão é um vértice, e o peso de cada aresta é a distância euclidiana do centroide de um símbolo até o centroide de outro símbolo.

2.12 Árvore Geradora Mínima

Árvore Geradora Mínima ou Minimum Spanning Tree (MST) é uma árvore que conecta todos os vértices de um grafo de forma que a soma dos pesos das arestas seja a menor possível. Existem diversos algoritmos para a construção de uma MST de um grafo. Os mais conhecidos são *Kruskal* e *Prim*. Resumidamente o algoritmo de *Prim* inicia a construção da MST com a escolha de um vértice aleatório (s_t) do grafo. Uma nova aresta (s_t, s_n) é adicionada à MST se o peso $w(s_t, s_n)$ for o menor de todas as arestas, onde s_t pertence à MST e s_n não.

Para cada novo vértice adicionado, o processo é repetido até que todos os vértices estiverem na MST (MATSAKIS, 1999).

Outro método para obtenção da árvore geradora mínima de um grafo é o algoritmo de Kruskal Gersting (2004). Enquanto o algoritmo de Prim é iniciado por um vértice qualquer, o algoritmo de Kruskal inicia pela aresta com menor peso, e adiciona as demais arestas em ordem crescente. As arestas não são adicionadas caso a inclusão da mesma resulta num ciclo. O algoritmo termina quando todos os vértices foram adicionados na árvore. Na sequência o pseudocódigo do algoritmo de Kruskal adaptado de Gersting (2004).

Algoritmo 1: ALGORITMO DE KRUSKAL

Entrada: Grafo G ,

Saída: Lista de arestas

```

1  início
2      Ordena arestas de  $G$  em ordem crescente;
3       $T = []$ ;
4      repita
5          se inclusão da próxima aresta não causa ciclo então
6              T.inclui(aresta);
7          fim
8      até  $T$  possuir todos os vértices de  $G$  e é conexo;
9  fim
10 retorna  $T$ ;
```

Neste trabalho a utilização de uma MST é utilizada para auxiliar na construção da árvore de *baselines* que será abordada na seção dos trabalhos relacionados.

3 Trabalhos Relacionados

O reconhecimento de expressões matemáticas requer muitas etapas e algumas delas, dependendo do método utilizado para resolvê-las, podem se tornar muito complexas. Por isso, muitos dos trabalhos feitos nessa área estão divididos focando em resolver apenas uma parte do problema. As três principais etapas são: **segmentação, reconhecimento e análise estrutural**. A maioria dos trabalhos encontrados concentram-se no reconhecimento *online*, que não é o foco deste trabalho. Porém, é possível explorar a etapa da análise estrutural pois nesta fase os símbolos já foram classificados, só resta mapear o relacionamento entre os símbolos e gerar uma estrutura de dados contendo a expressão. No reconhecimento *online* as etapas de segmentação e classificação estão fortemente ligadas, portanto, este trabalho não pode tirar proveito dos métodos desenvolvidos nessas etapas.

Vários dos trabalhos que concentram-se no reconhecimento *offline* realizam a classificação dos símbolos com bases de dados de expressões impressas. Normalmente as fontes tipográficas são desconexas, ou seja, os símbolos estão todos afastados, ao contrário dos símbolos manuscritos que podem se encostar. Por conta disso, nem todos os métodos de segmentação *offline* encontrados podem ser aplicados à segmentação de símbolos manuscritos. Porém, as etapas de extração de características e classificação dos trabalhos com fontes tipográficas são parecidas com as etapas do reconhecimento manuscrito.

Outro fator observado em vários trabalhos, foi a dificuldade de encontrar base de dados com expressões manuscritas com os dados já classificados. Muitos dos autores acabam gerando suas próprias bases de dados para testar seus algoritmos. As bases de dados dos trabalhos de reconhecimento *online* não podem ser utilizadas de forma direta para o reconhecimento *offline* pois não guardam todos os pontos dos símbolos, ao invés, guardam dados temporais e outras informações. Porém, segundo Muñoz (2015) é possível modificar essas bases de dados removendo essas informações adicionais e aplicando interpolação linear nos pontos gerando uma imagem muito próxima do que seria a escrita à mão.

As buscas dos trabalhos relacionados se deram principalmente pelas palavras-chaves: *offline recognition*, *handwriting* e *mathematical expression*. Os termos também foram buscados em português: reconhecimento *offline*, expressões matemáticas manuscritas, expres-

sões matemáticas escritas à mão. Algumas constatações a respeito das buscas:

- Buscas por reconhecimento manuscrito são encontrados resultados sobre reconhecimento *online*.
- Buscas por reconhecimento *offline* são encontrados resultados sobre reconhecimento de símbolos tipográficos (fontes de computador) e não manuscritos.
- Buscas usando os dois termos junto (manuscrito e *offline*), poucos resultados são encontrados que abordam as três etapas (segmentação, reconhecimento e análise estrutural).

Por não haver trabalhos completos sobre reconhecimento *offline* manuscrito, um trabalho envolvendo este escopo e abordando todas as etapas do reconhecimento de expressões, faz com que este trabalho seja relevante. A seguir são apresentados os métodos encontrados na literatura para cada uma das etapas do reconhecimento.

3.1 Pré-processamento

A imagem digital adquirida da expressão matemática deve ser submetida à processamentos que ajudam a eliminar informações desnecessárias da foto e facilitar os próximos passos: segmentação e reconhecimento. A seguir, uma lista de possíveis técnicas que foram utilizadas por alguns autores:

Redução de ruídos: aumentam a qualidade do documento. Aplicação de filtros de mediana e de média (DESAI, 2012).

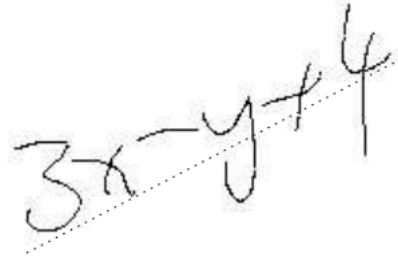
Binarização (*thresholding*): conversão de uma imagem em escalas de cinza para binário. Existem dois tipos de binarização: global e adaptativo (local) (VAMVAKAS, 2007).

Normalização da espessura dos traços: ajuda a reduzir a quantidade de informação da imagem (VAMVAKAS, 2007).

Skew correction: corrige o alinhamento do texto como um todo para que todas as linhas fiquem na mesma angulação. Técnica normalmente utilizada para reconhecimento de textos. Smithies (1999) investigou a utilização desta técnica para expressões matemáticas manuscritas e notou que após a correção de fórmulas com expoentes, as relações geométricas ficam ambíguas, dificultando encontrar as relações entre os símbolos. A figura 3.1 mostra uma expressão matemática não alinhada corretamente. Após aplicar a correção, o resultado obtido

foi $3x - y + 4$, porém, o resultado correto era 3^{x-y+4} . Pela dificuldade de corrigir o alinhamento da expressão sem perder as relações espaciais dos símbolos os autores normalmente não fazem esta correção.

Figura 3.1: *Skew correction*.



Fonte: Smithies (1999).

As principais técnicas aplicadas para correção do alinhamento são: *Linear Least Squares Regression*, *projection profiles* e *Hough Transformation*.

Slant removal: corrige a inclinação dos símbolos. A inclinação dos símbolos varia com a caligrafia da pessoa. Este passo é aplicado para normalizar todos os símbolos na mesma inclinação (GARAIN; CHAUDHURI, 2005). Principais técnicas utilizadas são: cálculo da média dos ângulos dos símbolos verticais próximos, *Bozinovic Srihari Method* e inclinação dominante utilizando entropia mínima de projeção vertical.

3.2 Segmentação

A técnica mais simples para a segmentação de símbolos em uma expressão matemática é a utilização da segmentação por **componentes conexos**, porém, quando aplicada a alguns símbolos matemáticos como $=$, $\leq$, i , j , entre outros, que são compostos por múltiplos componentes, esta técnica não agrupa esses tipos de símbolos (CHAN; YEUNG, 2000). Alguns autores deixam para agrupar estes símbolos em etapas posteriores (análise estrutural) (TAPIA, 2005).

Faure e Wang (1990) propuseram um sistema em dois módulos para segmentação de expressões. O primeiro módulo aplica projeção de histogramas em X e em Y para decidir como segmentar a expressão. Porém, quando aplicado em símbolos como “ $\sqrt{\quad}$ ”, esse algoritmo falha. Para contornar esse problema, uma máscara é utilizada para a sua remoção antes de aplicar as projeções. Com isso, todos os símbolos, inclusive os símbolos internos são segmentados. Como resultado do primeiro módulo, uma árvore da relação dos símbolos é re-

tornada. O segundo módulo é utilizado para corrigir a árvore agrupando símbolos com mais de um componente ($=, i, j$, etc).

Outro problema difícil de resolver na segmentação manuscrita é quando dois símbolos estão sobrepostos (um sobre o outro) (NAIK; METKEWAR, 2015). Para expressões com símbolos tipográficos, este problema é menos frequente. Okamoto e Miao (1991) propôs particionar uma expressão matemática com fontes tipográficas dividindo a expressão em componentes aplicando recursivamente o método *Projection Profile Cutting* (utiliza projeção de histogramas) nos eixos horizontal e vertical. Alguns passos adicionais precisam ser feitos para corrigir símbolos compostos por mais de um componente.

3.3 Extração de Características

Bharambe (2015) fez uma revisão na literatura sobre as características extraídas dos símbolos isolados de uma expressão matemática para que depois possam ser classificadas. Observou que os autores usaram técnicas comuns como *Zoning*, *Normalized Chain Code (NCC)*, momento invariante e projeção de histogramas. Essas técnicas são frequentemente usadas para compor o vetor de características utilizado na etapa de classificação *offline* de símbolos matemáticos.

Quadro 3.1: Tabela comparativa da taxa de reconhecimento de símbolos para cada *feature*

Features	Feature vector size	Recognition rate(%)	
		Letters	symbols
Zoning density + average + height+ width (16+8+2+2)	28	96	82
Normalized chain code(NCC)(16)	16	93	91
moment invariant+NCC(7+16)	23	95.5	98.2
Projection histogram	127	95	92
NCC+zoning + (d1, d2) +projection histogram(16+8+2+127)	153	99.9	95.6

Fonte: Bharambe (2015)

A Tabela 3.1 mostra uma comparação entre a taxa de reconhecimento para cada um dos vetores de características. O conjunto de dados utilizado possuía números, letras e símbolos matemáticos totalizando 3560 imagens de símbolos manuscritos. Foi utilizado como

classificador o *SVM*. Pode-se perceber que utilizando um vetor de apenas 23 *features* (7 de momento invariante e 16 de NCC) foi possível obter bons resultados: 95.5% para letras e 98.2% para os demais símbolos. Para conseguir resultados melhores na taxa de reconhecimento das letras, foi utilizado outra combinação de *features*: *NCC*, *zoning* e projeção de histogramas. Com essa nova combinação, a taxa de reconhecimento das letras foi de 99.9%, porém, com essas *features* o tamanho do vetor de *features* em relação a outra combinação de features subiu de 23 para 153 (aproximadamente 6.6 vezes).

3.4 Métodos de Classificação

Bage, Adhiya e Gharde (2010) pesquisaram publicações de diferentes técnicas de classificação usadas para o reconhecimento *online* e *offline* de expressões matemáticas. Além dos classificadores, os autores apresentaram quais técnicas de extração de características foram utilizadas em cada uma das publicações e a taxa de acerto dos experimentos. A Tabela 3.2 mostra as publicações, quais técnicas de extração características foram usadas e o tipo da *dataset*: *online*, *offline* (tipográfico ou manuscrito).

Quadro 3.2: Tabela com as publicações, técnicas de extração de características e a *dataset* usada

Paper Title	Feature Extraction Technique	Printed/ Handwritten
A Template Matching Distance for Recognition of On-Line Mathematical Symbols [13]	Chain Code	Online Handwritten Mathematical Symbols
Support Vector Machines for Mathematical Symbol Recognition [11]	Directional Features	Printed Mathematical Symbol
Comparing Several Techniques for Offline Recognition of Printed Mathematical Symbols[12]	Global Features, Local Features (Height, normalized grey level , horizontal grey-level derivative , vertical grey-level derivative and Gaussian function)	Offline Printed Mathematical Symbol
Evaluation of Feature Extraction and Classification Techniques on Special Symbols [14]	Moment Invariant, Gaber Feature	Offline Printed Mathematical Symbol
Prototype Pruning by Feature Extraction for Handwritten Mathematical Symbol Recognition[7]	Geometric Features, Ink Related Features, Directional Features, Global Features, Local Features	Online Handwritten Mathematical Symbols
Incorporating Contextual Character Geometry in Word Recognition [15]	Character Geometry	Handwritten Word Recognition

Fonte: Bage, Adhiya e Gharde (2010)

A Tabela 3.3 mostra os resultados obtidos de cada uma das publicações e quais foram os classificadores utilizados. As conclusões dos autores foram parecidas, em geral concordaram que o SVM obteve maiores taxas de acerto. Porém, não é correto comparar os resultados pois cada trabalho utilizou uma base de dados diferente.

Quadro 3.3: Tabela comparativa dos classificadores

Paper Title	Classification	Database Used	Result/Remark
A Template Matching Distance for Recognition of On-Line Mathematical Symbols [13]	Nearest Neighbour	90 symbols, 62100 total samples	98.94%
Support Vector Machines for Mathematical Symbol Recognition [11]	SVM	284,739 character symbols extracted from 363 journal articles	97.0%
Comparing Several Techniques for Offline Recognition of Printed Mathematical Symbols [12]	classical and novelty, SVM, HMM, WNN	UW-III database InfyCDB-1 database	The best results were obtained with SVM classifiers
Evaluation of Feature Extraction and Classification Techniques on Special Symbols [14]	SVM, HMM, Neural network	Scan pdf data set	99.91% recognition rate can be higher using SVM
Prototype Pruning by Feature Extraction for Handwritten Mathematical Symbol Recognition[7]	Elastic Matching	10,000 mathematical handwriting samples	94.8%
Incorporating Contextual Character Geometry in Word Recognition [15]	Segment Combination, Character Alignment	5735 US postal word images	96.80%

Fonte: Bage, Adhiya e Gharde (2010)

3.4.1 Redes Neurais Convolucionais

LeCun (1998) projetou uma CNN chamada LeNet-5 e obteve sucesso para o reconhecimento de dígitos utilizando a *dataset* MNIST. LeNet-5 obteve erros de menos de 1% chegando próximos a outros métodos como SVM que a escolha é explícita dos descritores de características (LECUN, 1998). LeNet-5, por ser uma CNN, tem a vantagem de não precisar extrair características. Além disso, a LeNet-5 é invariante à translação, escala, rotação, *squeezing*, largura dos traços dos caracteres e à vários tipos de ruídos. O trabalho de LeCun (1998) em reconhecimento de dígitos é até hoje considerado referência neste tipo de reconhecimento (FERNANDES, 2013).

A desvantagem de utilizar uma CNN é o tempo de treinamento. Embora seja mais rápido que uma MLP, ainda é considerado lento se comparado com classificadores como SVM. Portanto, é desejável que o treinamento desse tipo de RNA seja feito via GPU (NIELSEN, 2015).

Funções de ativação que vêm sendo utilizadas nas arquiteturas das redes neurais convolucionais para aplicações visuais são Rectified Linear Unit (ReLU) (GLOROT; BORDES; BENGIO, 2011) e *softmax* (DUNNE; CAMPBELL, 1997). A ativação ReLU é definida como $f(x) = \max(0, x)$, onde x é a entrada de um neurônio. Já a função de ativação *softmax*, normalmente utilizada na última camada da CNN (GLOROT; BORDES; BENGIO, 2011), é usada para “afunilar” e distribuir as probabilidades para cada uma das classes do problema. Segundo Nielsen (2015) a função de ativação *softmax* pode ser definida como:

$$a_j^L = \frac{e^{z_j^L}}{\sum_k e^{z_k^L}} \quad (3.1)$$

A notação a_j^L representa a ativação do j -ésimo neurônio da L -ésima camada (*layer*). Já z_j^L é a soma de todos os pesos das arestas que chegam em um neurônio multiplicado pela ativação dos neurônios da camada anterior. Nielsen (2015) define z_j^L como:

$$z_j^L = \sum_k w_{jk}^L a_k^{L-1} + b_j^L \quad (3.2)$$

A variável w_{jk}^L representa o peso da aresta que parte do k^{th} neurônio na $(l-1)^{th}$ camada até o j^{th} neurônio da l^{th} . Por último b_j^L representa o *bias*. Esta notação é útil pois permite a fácil visualização da rede neural como uma matriz de pesos onde as colunas são as camadas da rede.

Juntamente com a ativação *softmax*, é normalmente utilizada a função de erro *cross-entropy* (DUNNE; CAMPBELL, 1997) para calcular a “diferença” do vetor de probabilidades reais de uma instância do conjunto de treino com o vetor previsto pela CNN. Nielsen (2015) define *cross-entropy* como:

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)] \quad (3.3)$$

Na equação acima, a é a saída de um neurônio, n é número total de itens no conjunto de treinamento, a soma acontece sobre todos os itens do conjunto de treinamento (x), e y corresponde a saída desejada.

A minimização da função de erro *cross-entropy* pode ser feita com o método *Gradient Descent* (GD) (NIELSEN, 2015). A aplicação deste método durante o treinamento juntamente com *backpropagation* permite atualizar os parâmetros da rede (pesos) que diminuam o erro do classificador perante as instâncias do conjunto de treinamento. A cada atualização dos parâmetros, o GD passa por todas as instâncias do conjunto de treino. Uma versão modificada do GD é o Stochastic Gradient Descent (SGD) em que a cada atualização dos parâmetros só precisa passar por uma ou algumas instâncias do conjunto de treino (MURPHY, 2012). Este subconjunto de instâncias é escolhido aleatoriamente. Outro método de otimização que vem sendo utilizado em CNNs é o Adaptive Moment Estimation (ADAM), uma adaptação do SGB (KINGMA; BA, 2014). A utilização deste método é adequada quando a arquitetura da rede neural é grande (possui muitos parâmetros) e o conjunto de treino possui muitas instâncias (KINGMA; BA, 2014).

Uma técnica utilizada para reduzir as chances de *overfitting* é o *dropout* Srivastava (2014). Esta técnica “desliga” uma porcentagem dos neurônios da rede. A cada *epoch* (iteração do treinamento), são escolhidos neurônios aleatórios para serem “desligados”. Desta forma, evita-se que certos neurônios da rede se especializem em apenas uma, ou poucas classes. Com essa abordagem, nenhum neurônio das camadas escondidas são “estáveis” Srivastava (2014), e ao final do treinamento, as chances de que todos os neurônios tenham influências sobre todas as classes são maiores.

3.5 Geração da Árvore de Expressões Matemáticas

Sistemas existentes de reconhecimento de expressões matemáticas podem ser classificados em duas categorias: a primeira, chamada de **Análise Estrutural**, utiliza métodos para obter a representação estrutural diretamente pela análise dos pixels ou traços das imagem. A segunda categoria, utiliza gramáticas formais e é chamada de **Mathematics Notation Parsers**. Abaixo segue uma classificação feita por Zanibbi et al. (2001) e modificada pelo autor deste trabalho de alguns métodos encontrados em cada uma das categorias:

1. Análise Estrutural

- *Projection Profile Cutting* (OKAMOTO; MIAO, 1991)
- *Baseline Structure Tree* (ZANIBBI; BLOSTEIN; CORDY, 2001)
- *Minimum Spanning Tree* (MATSAKIS, 1999)

2. Mathematics Notation Parsers

- *Graph Grammars* (GRBAVEC; BLOSTEIN, 1995)
- *Stochastic Grammars* (CHOU, 1989)
- *Procedural Translation* (LEE; LEE, 1994)
- *Structure Specification Schemes* (CHANG, 1970)
- *Attributed String Grammars* (ANDERSON, 1967)

Os autores citados acima foram os pioneiros em cada um dos métodos. Algumas dessas técnicas continuam a serem utilizadas com alguns aperfeiçoamentos, e algumas delas, são combinadas (MUÑOZ, 2015). Alguns métodos foram feitos pensando para serem aplicados

em reconhecimento *online* e outros *offline*. Alguns dos métodos *online* podem ser testados no reconhecimento *offline* com algumas modificações. Nas próximas subseções, alguns dos métodos citados serão explicados em maiores detalhes.

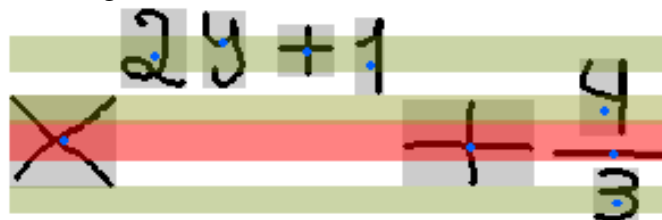
3.5.1 *Baseline Structure Tree*

Zanibbi, Blostein e Cordy (2001) propuseram um método de análise estrutural chamado *Baseline Structure Tree* (BST). O objetivo desse método é construir a árvore, que representa a expressão matemática, através de um conceito chamado *baselines*. Na matemática, *baseline* têm dois significados: o primeiro, uma linha imaginária que passa pelos símbolos da expressão que estão adjacentes horizontalmente, e o segundo, um conjunto de símbolos que estão horizontalmente adjacentes. Uma BST representa a estrutura espacial entre os conjuntos de símbolos horizontalmente adjacentes (ZANIBBI; BLOSTEIN; CORDY, 2001).

O resultado final da etapa do reconhecimento dos símbolos (classificação) é uma lista contendo os símbolos já identificados (rotulados) e o *bounding box* ($Xmin$, $Ymin$, $Xmax$, $Ymax$) de cada símbolo. A primeira etapa do BST é ordenar esta lista pelo $Xmin$ de cada símbolo de forma crescente. Esta etapa é realizada porque a ordem de leitura de uma expressão matemática acontece da esquerda para a direita como pode ser observado no trabalho de Costagliola (1997) sobre escaneamento de linguagens visuais.

O segundo passo do BST é a criação da *baseline* principal da expressão matemática. Essa *baseline* é representada por uma lista de símbolos que possuem o centroide na mesma região dos limiares do primeiro símbolo da lista (o mais esquerdo da expressão). A figura 3.2 mostra a *baseline* principal na cor vermelha e as demais *baselines* em outra cor. Também possível observar o *bounding box* de cada símbolo na cor cinza e o centroide em azul. A expressão matemática que essa figura representa é: $x^{2y+1} + \frac{4}{3}$.

Figura 3.2: Regiões de alcance do símbolo e limiares do centroide.



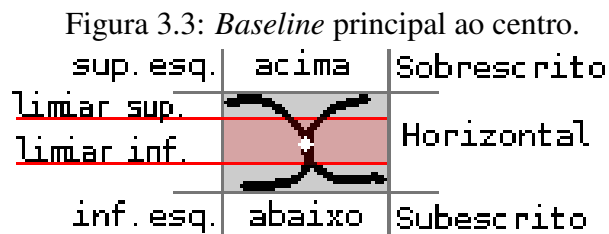
Fonte: produzido pelo próprio autor.

A *baseline* principal da figura 3.2 é representada pelo conjunto $(x, +, -)$. Note que

a barra horizontal não tem distinção se é o símbolo de menos ou a barra da divisão. As outras *baselines* são: $(2, y, +, 1)$, (4) e (3) .

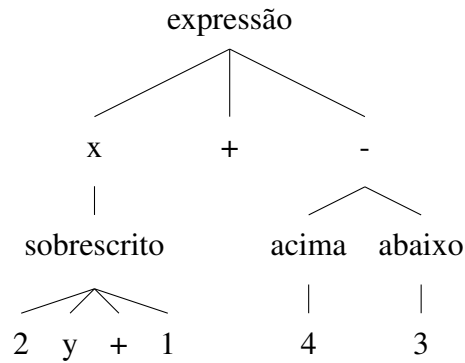
Após identificar a *baseline* principal o algoritmo precisa encontrar a relação dos símbolos da *baseline* principal com os demais símbolos da expressão. Essas relações são encontradas aplicado o conceito de **dominância** nas regiões de alcance de cada símbolo (ZANIBBI; BLOSTEIN; CORDY, 2001). Para cada região analisada que possuem símbolos o algoritmo procura recursivamente pelas outras *baselines*.

Na figura 3.3 é mostrado o símbolo x e sua região de alcance. Além disso, são mostrados em linhas vermelhas os limiares que são influenciados pelo *bounding box*, pelo centroide e pela grafia do símbolo (ascendente, descendente e central). Neste caso, como o x é classificado como grafia central (regular), o centroide está posicionado exatamente ao centro do *bounding box*. Porém, se a grafia for ascendente a coordenada y do centroide é posicionada em $\min Y + \frac{1}{4}(\max Y - \min Y)$. Se a grafia for descendente a coordenada y do centroide é posicionado em $\min Y + \frac{3}{4}(\max Y - \min Y)$. Essa atribuição é feita com o objetivo de normalizar a localização da *baseline* para cada tipo de grafia (ZANIBBI; BLOSTEIN; CORDY, 2001). Lembrando que a coordenada x do centroide é sempre na metade do *bounding box*.



Fonte: produzido pelo próprio autor.

A construção do restante da árvore parte da *baseline* principal. O caminho da raiz até uma folha é formado por regiões e símbolos alternados. Essas regiões representam a relação de dominância de um símbolo com um ou mais símbolos. Após encontrar as relações de dominância uma árvore de *baselines* é formada.



Para o caso da árvore acima já é possível fazer uma conversão para código LaTeX que produzirá a expressão correta. Porém, a árvore ainda não está completa porque ainda não foi identificado na estrutura as subárvores da soma (na *baseline* principal) nem da operação de multiplicação no expoente. Após montar a BST, o algoritmo proposto por Zanibbi, Blostein e Cordy (2001) aplica análise léxica, sintática e semântica para corrigir possíveis erros na BST. Na análise léxica é possível agrupar símbolos compostos por um ou mais traços, por exemplo, o sinal de igualdade (=) ou "menor ou igual que" ($\leq$), nome de funções (*ln*, *lg*, *log*, *exp*, *sin*, *cos*, *tan*), agrupamento de uma sequência de dígitos em um número, números decimais (1.00, 0.01, .01), entre outros. A análise sintática converte a BST para um outro tipo de árvore para que possa ser verificado em uma gramática formal se a expressão é válida. Por fim, a análise semântica infere operadores (operandos adjacentes representa a multiplicação de ambos), reorganiza os operadores para garantir a ordem de precedência, entre outros.

A correção na árvore é feita através de uma linguagem chamada TXL capaz de realizar transformações em árvores. Com essa linguagem são definidos quais tipos de transformações devem ocorrer. Primeiramente é feita uma busca da raiz da árvore até as folhas buscando por padrões pré-definidos e então são aplicadas as transformações. A árvore resultante pode ser convertida para LaTeX ou convertida em um formato textual para poder ser utilizada em programas algébricos computacionais.

O método de Zanibbi, Blostein e Cordy (2001) foi feito para reconhecer expressões matemáticas com fontes tipográficas. O autor fez alguns testes com expressões manuscritas e obteve resultados razoáveis. Porém, percebeu alguns problemas no reconhecimento da *baseline* principal e também quando são utilizados dois somatórios seguidos como pode ser visto na quarta fórmula da figura 3.4.

Figura 3.4: Método BST aplicado em expressões manuscritas.

$\left(\frac{\sqrt{ab+b}}{3}\right)^2$	$\left(\frac{\sqrt{ab+b}}{3}\right)^2$
$\overline{a \vee b \vee c}$	$\overline{a \vee b \vee c}$
$\sum_{i=100}^{7426} i + \cos n$	$\sum_{i=100}^{7426} i + \cos n$
$\bar{A}B \sum_{i=1}^2 \sum_{j=2}^2 i j$	$\bar{A}_B \sum_{i=1}^2 \sum_{j=2}^2 i j$
$\frac{\sqrt{\frac{4x^2}{\sqrt{2a}}} y^2 + \frac{3}{4}}{\left(\int_{2b}^3 \frac{x^2}{4+a} dx\right)^3} x^2 + \frac{-b + \sqrt{b^2 - 4ac}}{2a}$	$\frac{\sqrt{\frac{4x^2}{\sqrt{2a}}} 4^{2^3+4}}{\left(\int_{2b}^3 \frac{x^2}{4+a} dx\right)^3 x^2} + \frac{-b + \sqrt{b^2 - 4ac}}{2a}$

Fonte: Zanibbi, Blostein e Cordy (2001).

3.5.2 Árvore Geradora Mínima

A utilização de árvore geradora mínima ou *minimum spanning tree (MST)*, proposta por Matsakis (1999), começou a ser usada com o intuito de agrupar símbolos que foram escritos com mais de um traço (por exemplo, '='). O sistema foi projetado para o reconhecimento *on-line*. Para geração da estrutura de dados foi utilizado *geometric grammar* que é uma abordagem híbrida: um *parser* para criar a estrutura juntamente com informações da análise estrutural para a localização de símbolos que estão nas regiões de alcance de outros símbolos, por exemplo: subscrita e sobrescrita. A MST foi apresentada como uma etapa posterior a classificação e anterior a etapa da geração da árvore de expressões.

Para construir a árvore geradora mínima de uma expressão matemática, primeiramente é criado um grafo de todos os símbolos da expressão. Esse grafo é completo, ou seja, todos os símbolos estão conectados com todos. Os pesos das arestas variam conforme o problema que está tentando ser resolvido. A MST pode ser construída desse grafo pelo algoritmo de Prim (abordado na Seção 2.12).

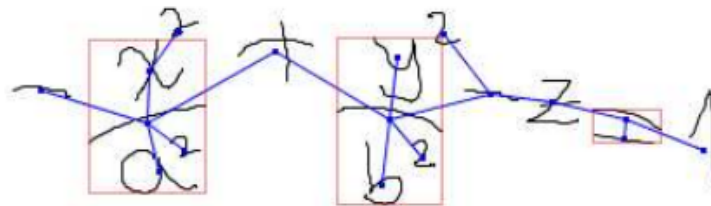
Tapia (2005) estendeu o conceito do Matsakis (1999) e passou a utilizar MST du-

rante a construção da árvore. Além disso, baseou-se nos trabalhos de Zanibbi, Blostein e Cordy (2001) com a BST e dominância. Essa abordagem utiliza apenas análise estrutural para a construção da árvore.

Durante a criação da BST, Tapia (2005) cria três MSTs. Uma antes da construção e duas enquanto a BST está sendo construída. A primeira MST é utilizada para corrigir a construção irregular dos *layouts* horizontais em expressões manuscritas. A segunda MST é a principal e é usada para agrupar símbolos compostos por vários componentes (desconexos ou não). A terceira MST é usada para encontrar as linhas de uma matriz.

Tapia (2005) utilizou MST com o objetivo de corrigir erros da BST. A BST proposta por Zanibbi, Blostein e Cordy (2001) considerava que o reconhecimento fosse de símbolos tipográficos, por isso, a construção da *baseline* principal sempre era correta. Tapia (2005) aplicou para o reconhecimento *online* manuscrito e percebeu que com as regularidades da escrita à mão, a *baseline* principal não era construída corretamente.

Figura 3.5: Irregularidades do *layout* horizontal.

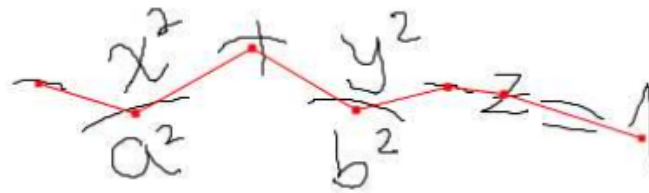


Fonte: (TAPIA, 2005).

A construção da *baseline* principal consiste em encontrar símbolos que descrevam arranjos horizontais na expressão. Tapia (2005) complementou a ideia de Zanibbi, Blostein e Cordy (2001) para a construção desses arranjos definindo o conceito de "direita horizontal" (*right horizontal*). Para formalizar o conceito, sejam s e s_r , onde s_r é o símbolo vizinho horizontal que está na região de abrangência da direita de s ou que s está na região de abrangência esquerda de s_r . Portanto, o arranjo horizontal é composto pelos símbolos que dominam outros símbolos pela direita ou pela esquerda e que os seus centroides estão dentro dos limiares do primeiro símbolo à esquerda da expressão. A figura 3.5 mostra irregularidades da escrita à mão que dificultam a construção da *baseline* principal seguindo essa definição de arranjo horizontal. Observando a imagem citada anteriormente, é possível perceber que o sinal de mais (+) está na região super-escrita do símbolo da barra de divisão, e portanto, a *baseline* principal será construída de forma errada. Normalmente os problemas causados na identificação correta da *baseline* principal acontecem pela presença da barra horizontal da fração. Para corrigir esse

problema, Tapia (2005) propôs a construção de uma MST especificamente para identificar e corrigir problemas com barras horizontais. Para cada barra horizontal encontrada, se a barra domina outro símbolo em uma de suas regiões, o peso da aresta da barra até este símbolo é a distância mínima entre os pontos de atração. Se a barra horizontal não dominar este símbolo, o peso será a distância mínima dos pontos de atração da barra até o centroide do outro símbolo. Se nenhum dos dois símbolos que estão sendo analisados for uma barra horizontal, o peso da aresta é a distância dos centroides dos dois símbolos. Depois que a MST é construída, são atualizados os valores do *bounding-box* de todos os símbolos de fração (barra horizontal) para abrigar os símbolos que estão acima e abaixo da barra. A figura 3.5 mostra a MST construída e os *bounding-boxes* em vermelho atualizados. Com os *bounding-boxes* atualizados, os limiares do símbolo também são atualizados, e agora, a relação da "direita horizontal" pode ser aplicada nessa expressão. A figura 3.6 mostra a *baseline* dominante construída.

Figura 3.6: *Baseline* dominante construída.



Fonte: (TAPIA, 2005).

A segunda MST construída é a principal. O peso das arestas é atribuído da seguinte forma:

- Caso não exista relação de dominância entre dois símbolos, o peso da aresta é a distância euclidiana dos centroides dos símbolos.
- Caso exista relação de dominância, o peso da aresta é a menor distância entre seus pontos de atração.

Tapia (2005) define pontos de atração (*attractor points*) como pontos localizados nas bordas do *bounding-box* de cada símbolo. O número e a posição desses pontos dependem da classe (aspecto comportamental) de cada símbolo.

Com a MST principal construída é possível verificar se existem símbolos que agrupados representam outros símbolos aceitos pelo LaTeX, por exemplo: $\sin$, $\cos$, $\tan$, $\lim$, $\leq$ etc. Caso sejam encontrados, é criado um novo símbolo com os novos atributos (rótulo, centroide, *bounding-box*, limiares etc) e os símbolos antigos são removidos da árvore.

Um problema identificado por Tapia nesta etapa, é que alguns operandos podem estar muito distantes de seus operadores. Isso pode gerar erros e construir uma expressão inválida. Para solucionar esse problema, Tapia (2005) usou um parâmetro α que multiplica o peso das arestas dos operadores e seus argumentos. Esse parâmetro é definido como $0 < \alpha < 1$. Tapia (2005) usou $\alpha = \frac{1}{4}$ e obteve bons resultados.

A terceira MST é utilizada para auxiliar na construção de matrizes através da identificação dos símbolos ']', '[' e das linhas da matriz. Porém, a criação de matrizes foge do escopo deste trabalho, portanto, não será abordado.

3.6 Geração do LaTeX

A conversão da árvore para código LaTeX depende de quais métodos são utilizados para construir a árvore. Considerando que a árvore da expressão está no formato de *baselines* (um nível símbolo e outro região alternados) proposto por Zanibbi et al. (2001), basta percorrer a árvore em pré-ordem e criar uma *string* substituindo as regiões da árvore pela notação correspondente do LaTeX. Uma forma de fazer isso, é criar mapeamento que relaciona a notação utilizada na árvore com a notação do LaTeX.

O trabalho de Sider, Ozahata e Hirata (2010) rotulou os símbolos na etapa da classificação já com a notação LaTeX. Construiu o grafo da expressão com MST de forma que cada símbolo (vértice) possui no máximo oito arestas, uma para cada região de alcance dos símbolos (acima, abaixo, sobrescrito, subscrito, etc). Como os símbolos e operadores já estavam identificados pela notação LaTeX, basta percorrer a MST e escrever os símbolos encontrados e seus argumentos na *string* LaTeX.

3.7 Considerações

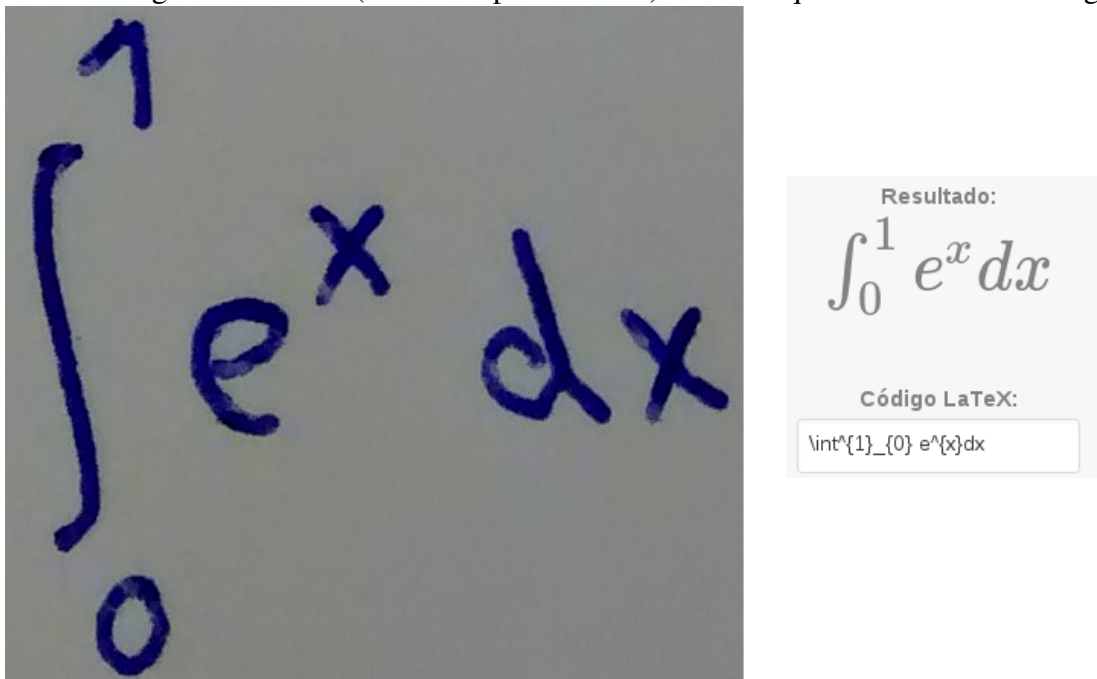
Nenhum dos trabalhos relacionados apresenta a implementação de todas as etapas de um reconhecedor *offline* manuscrito. Portanto, a sequência de métodos explorados pertencem a diversos trabalhos. Nem todos os métodos foram implementados da maneira que foram descritos pelos autores. Na etapa de classificação dos símbolos, a implementação teve como base o classificador CNN apresentado por LeCun (1998). A arquitetura da CNN partiu do trabalho de LeCun (1998) e foi adaptada para o reconhecimento de símbolos matemáticos. Já

na etapa de análise estrutural, a implementação foi baseada no trabalho de Zanibbi, Blostein e Cordy (2001) e principalmente no trabalho de Tapia (2005). As principais mudanças foram na construção do grafo em que os pesos das arestas foram calculados de maneira diferenciada. Outra grande diferença foi a construção da MST, que é feita antes de chamar a construção da BST. No trabalho de Tapia (2005), as MSTs são construídas a cada recursão da construção da BST.

4 Geração Automática de Código LaTeX a Partir de Expressões Matemáticas Manuscritas

Neste trabalho foi desenvolvido uma aplicação *web* em que o usuário pode fazer *upload* de uma imagem ou tirar uma foto via *webcam*. Esta imagem ou foto, deve conter uma expressão matemática manuscrita. A imagem é enviada ao servidor da aplicação. Ao receber esta imagem, o *backend* interpreta e gera o código LaTeX da expressão matemática. A *string* LaTeX é retornada ao *frontend* e o navegador renderiza a expressão matemática em uma fonte legível e mostra o código LaTeX que representa a expressão. A figura 4.1 mostra um exemplo de imagem de entrada e o resultado obtido após o sistema interpretá-la.

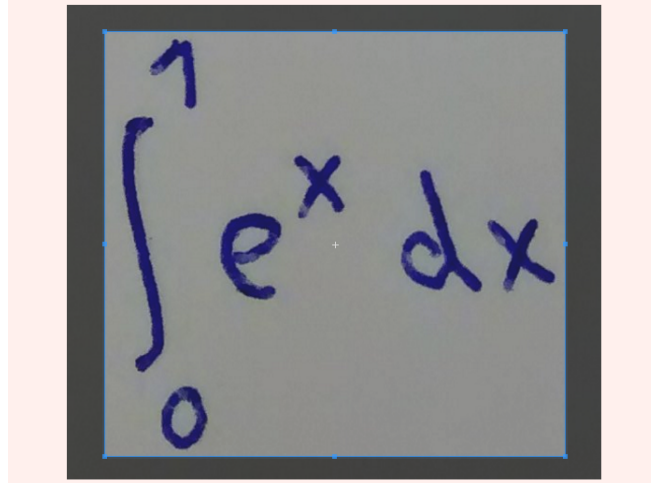
Figura 4.1: Imagem de entrada (fornecida pelo usuário) e a saída que é mostrada no navegador.



Produzido pelo autor.

Com o objetivo de ajudar o processamento, antes de enviar a imagem para o servidor, o usuário da aplicação utiliza uma ferramenta para localizar a expressão matemática na imagem através de uma ferramenta de seleção e corte (*crop*). Desta forma o usuário poderá eliminar componentes indesejáveis da imagem. A figura 4.2 ilustra a utilização da ferramenta de corte.

Figura 4.2: Ferramenta de seleção e corte.
Selecione somente a região da expressão matemática



Produzido pelo autor.

Após a requisição chegar ao servidor, a imagem passa por um sistema reconhecedor de expressões matemáticas. Este sistema reconhecedor é implementado em seis etapas: **pré-processamento**, **segmentação**, **pós-segmentação**, **classificação**, **análise estrutural** e **conversor LaTeX**. A etapa de análise estrutural deste trabalho foi baseada na tese de Tapia (2005), mas foi implementada com algumas modificações.

Figura 4.3: Etapas do reconhecedor de expressões matemáticas manuscritas.



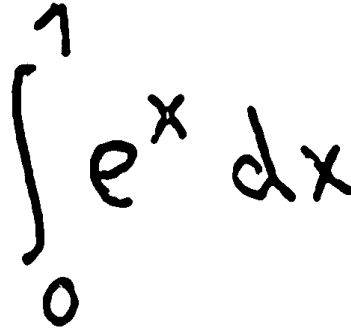
Produzido pelo autor.

4.1 Pré-processamento

Na primeira etapa a imagem é convertida para tons de cinza e logo em seguida é aplicado o filtro de mediana com máscara 7×7 para a redução de ruídos da imagem. Depois é aplicado a binarização (limiarização) convertendo a imagem para preto e branco. O objetivo da limiarização nesta etapa é deixar o fundo da imagem branco e a escrita em preto. A implementação da limiarização foi feita com método *Adaptative Mean Thresholding* fornecido pela biblioteca OpenCV com a função `adaptiveThreshold()` com os parâmetros `blockSize = 19` e

$C = 5$. Estes valores foram escolhidos empiricamente baseados em testes feitos com a escrita da expressão matemática feita em folhas brancas e amarelas, caneta cor azul e lapiseira grafite 0.5mm. Veja o resultado após limiarização na figura 4.4

Figura 4.4: Imagem de entrada após a binarização.

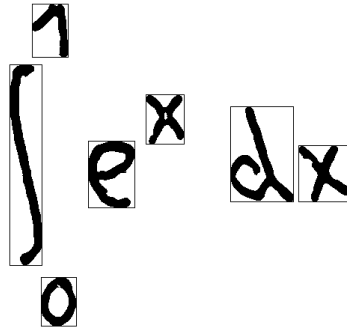


Produzido pelo autor.

4.2 Segmentação

A etapa da segmentação gera uma lista de imagens com todos os símbolos isolados. Para segmentar os símbolos é aplicado o algoritmo de componentes conexos explicado na fundamentação deste trabalho (Seção 2.7.2). O algoritmo atribui rótulos (números) aos componentes encontrados na imagem. Depois disso, são exportados para novas imagens os símbolos que possuem os mesmos rótulos. Este método não irá segmentar corretamente símbolos compostos por componentes desconexos, por exemplo: $=$, i , etc. Porém, na etapa da análise estrutural este problema é corrigido. Para a implementação da segmentação, foi utilizado a função *findContours()* fornecida pela biblioteca OpenCV que traz uma implementação otimizada para encontrar contornos e componentes em imagens binárias. A figura 4.5 apresenta os componentes encontrados na imagem.

Figura 4.5: Componentes encontrados.



Produzido pelo autor.

4.3 Pós-segmentação

A etapa de pós-segmentação é responsável por adaptar a imagem de cada um dos componentes encontrados para que sejam reconhecidos pelo classificador. O classificador espera uma imagem de tamanho 32×32 , porém, os componentes (símbolos) extraídos da etapa de segmentação possuem tamanhos variados. Uma opção para resolver esse problema é simplesmente redimensionar cada componente para o tamanho que o classificador exige. Porém, se os símbolos forem muito maiores que o tamanho exigido, ao redimensioná-los, os traços da imagem poderão sumir. Por isso, foi necessário realizar algumas alterações na imagem antes de ser redimensionada, sendo a principal delas, a dilatação dos traços se a imagem tiver um tamanho maior que um limiar. A figura 4.6 mostra os símbolos alterados e redimensionados para o tamanho 32×32 . O pseudocódigo com todos os detalhes pode ser encontrado na seção B.1 dos anexos.

Figura 4.6: Componentes alterados e redimensionados.



Produzido pelo autor.

4.4 Reconhecimento

A etapa de reconhecimento recebe a lista dos símbolos da pós-segmentação para classificá-los. Muitos autores utilizam métodos para extrair características da imagem antes de classificar, porém, neste trabalho, não foi necessário. O classificador utilizado foi Redes Neurais Convolucionais (CNN) que durante o processo de treinamento do modelo encontra as

features (características). Portanto, este método tem extração implícita de características. A escolha deste classificador foi devido a essa seleção implícita. Pelo trabalho de (LECUN, 1998) utilizando CNN, construiu um reconhecedor de caracteres com invariância à rotação, translação, escala, largura dos traços dos caracteres, ruídos, entre outros. Encontrar métodos de extração com todas essas peculiaridades demanda tempo e muita experimentação, enquanto que o CNN extrai as *features* automaticamente.

Para o treinamento da CNN foi necessário uma base de dados (*dataset*) com os símbolos já rotulados, isto é, identificados. A base de dados utilizada neste trabalho foi uma composição de quatro bases de dados. A primeira, composta por diversos símbolos matemáticos, é a HWRT (THOMA, 2015). Esta *dataset* é para reconhecimento *online* e por isso a informação dos símbolos não é uma imagem, mas um conjunto de pontos espaçados com informações temporais (*timestamp*) e informações sobre os traços (*pen-up* e *pen-down*). Como este trabalho é sobre reconhecimento *offline* (imagem), foi necessário converter os dados *online* para *offline* interpolando os pontos e gerando uma imagem PNG para cada símbolo. Além disso, a dimensão das imagens foi reduzida para 32×32 . A segunda *dataset* utilizada foi a Chars74K (CAMPOS; BABU; VARMA, 2009) composta de imagens de dígitos, alfabeto maiúsculo e minúsculo. As únicas modificações feitas nessa base de dados foram a dilatação dos traços com nível de dilatação dependendo da dimensão da imagem, e o redimensionamento da imagem para 32×32 . A terceira base foi a MNIST (LECUN, 1998) composta por imagens de dígitos (0 – 9) manuscritos. A modificação feita nessa base foi a inversão dos tons de cinza e o redimensionamento. A última base de dados foi criação do próprio autor para atender os símbolos matemáticos não encontrados nas demais bases. Para mais informações sobre as *datasets* ver apêndice A.

Devido as *datasets* não terem os dados rotulados corretamente em todos os dados, não foi selecionado o alfabeto inteiro. A dataset final possui 27 classes: dígitos 0 – 9; os caracteres $a, b, c, d, e, f, x, y, z$; os delimitadores (e); os operadores $+, -, \int, \sum, \sqrt{}$; e por fim, o símbolo de um ponto. Ao total a dataset possui 66271 símbolos. O conjunto foi dividido em 60% treinamento, 20% validação e 20% teste. A distribuição foi feita aleatoriamente, porém, de maneira balanceada entre as classes.

Para o treinamento, todas as imagens (em tons de cinza) foram normalizadas com os valores dos pixels de $[0, 255]$ para $[0.0, 1.0]$. A arquitetura da CNN foi projetada de maneira linear. A tabela 4.1 mostra arquitetura da CNN. As quatro primeiras camadas representam a etapa de extração de características. As próximas camadas representam uma rede neural MLP totalmente conectada. A utilização da técnica de *dropout* foi feita para reduzir as chances de

Quadro 4.1: Arquitetura da CNN

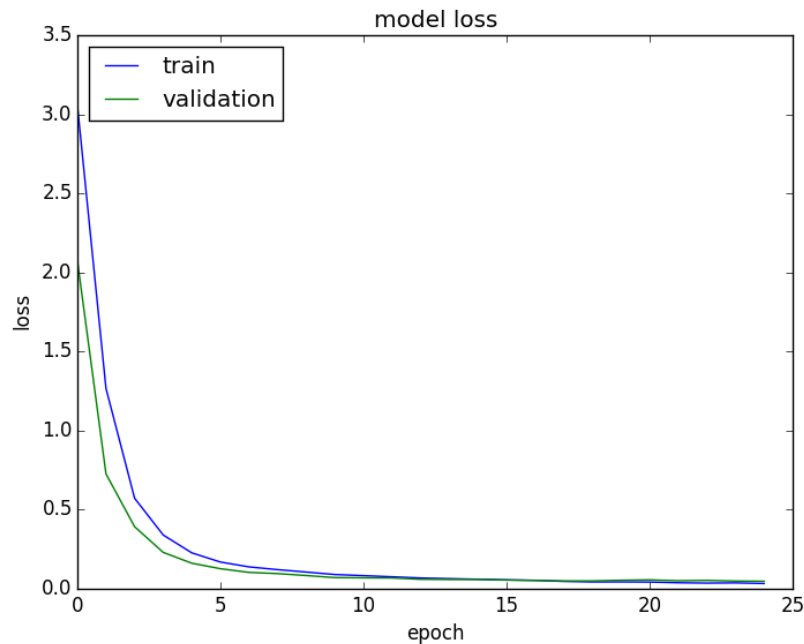
Entrada: imagem 32×32 normalizada em $[0.0, 1.0]$
camada 1: 32 <i>features maps</i> de convolução 5×5 com <i>stride</i> = 1
Ativação com Rectified Linear Unit (ReLU)
camada 2: <i>Pool</i> (2×2)
camada 3: 15 <i>features maps</i> de convolução 3×3 com <i>stride</i> = 1
Ativação ReLU
camada 4: <i>Pool</i> (2×2)
<i>Dropout</i> de 20%
camada 5: Conversão de matriz para vetor
camada 6: 256 neurônios totalmente conectados com a camada anterior
Ativação ReLU
camada 7: 128 neurônios totalmente conectados com a camada anterior
Ativação ReLU
camada 8: 27 neurônios totalmente conectados com a camada anterior
Ativação <i>softmax</i>
Saída: vetor com 27 elementos variando de $[0.0, 1.0]$

ocorrer *overfitting*.

Todos os pesos da CNN foram inicializados aleatoriamente seguindo a distribuição gaussiana. A função de erro (*loss function*) utilizada foi a *cross-entropy* ou *log loss* e o algoritmo otimizador utilizado foi o ADAM (versão modificada do Gradiente Descendente Estocástico). A otimização dos pesos da CNN é feita pelo otimizador ADAM minimizando a função de erro *cross-entropy* juntamente com *backpropagation*. O número de iterações inicialmente foi 50, porém, como pode ser observado na figura 4.7, a partir da 25ª iteração a convergência estabilizou. Por isso, o treinamento foi repetido com apenas 25 iterações. Toda a arquitetura, assim como os parâmetros, foram definidas empiricamente. Após a etapa de treinamento, o classificador foi avaliado através conjunto de testes que possui aproximadamente 13254 símbolos. O classificador obteve uma taxa média de erro de 1.407% em três execuções de treinamento, e o desvio-padrão foi de ± 0.061 . Cada execução do treinamento demorou aproximadamente 27 minutos em um processador Intel Core i7 4790K, e nas três execuções aproximadamente 81 minutos. A figura 4.7 mostra o decaimento da função de erro (*loss*) ao longo das iterações (*epoch*). Na mesma imagem, pode ser observadas duas linhas, uma azul e outra verde. Uma mostra o

decaimento em relação ao conjunto de treinamento e outra em relação ao conjunto de validação. Durante o treinamento é aplicada a técnica *cross-validation*, onde são criados subconjuntos mutuamente exclusivos do conjunto de validação para serem alternados com subconjuntos do conjunto de treino. Esta técnica é útil para saber o quão preciso é o modelo treinado.

Figura 4.7: Erro no decorrer das iterações do treinamento.



Produzido pelo autor.

Cada símbolo é classificado um por vez. O vetor de entrada para a CNN é a própria imagem com os pixels normalizados (neste caso pode ser visto como matriz de entrada). Após a CNN classificá-la, o símbolo é rotulado com a classe correspondente de maior probabilidade do vetor resultante da CNN. Depois de classificar todos os símbolos, um vetor é retornado. Este vetor além de possuir os símbolos rotulados, possui o *bounding-box* ($X_{min}, X_{max}, Y_{min}, Y_{max}$) de cada símbolo. Esta informação é encontrada depois que os símbolos foram segmentados e é referente as coordenadas da imagem original.

4.5 Análise Estrutural

O objetivo da etapa de análise estrutural é construir uma estrutura de dados (árvore) que represente a expressão matemática. Para construir a árvore foram implementados, com algumas modificações, os metodos de análise estrutural propostos por Zanibbi, Blostein e Cordy (2001) com *Baseline Structure Tree (BST)* e Tapia (2005) com *Minimum Spanning Tree*

Quadro 4.2: Definição dos centroides

central	$centroide_y = y + \frac{altura}{2}$
ascendente	$centroide_y = y + 0.66 \times altura$
descendente	$centroide_y = y + 0.33 \times altura$

Quadro 4.3: Definição dos limiares superior e inferior

central	$limiar_superior = y + 0.2 \times altura$
ascendente	$limiar_inferior = y + 0.8 \times altura$
descendente	

(*MST*). Após receber o vetor criado pela etapa de classificação é necessário identificar a grafia dos símbolos (**ascendentes**, **descendentes** e **centrais**). Mais informações sobre a grafia dos símbolos pode ser encontrada na seção 2.10.1. O próximo passo é encontrar o centroide. A coordenada x é sempre na metade do *bounding-box*. A coordenada y do centroide, varia conforme a grafia. Os valores foram baseados na tese de Tapia (2005). Veja a tabela 4.2

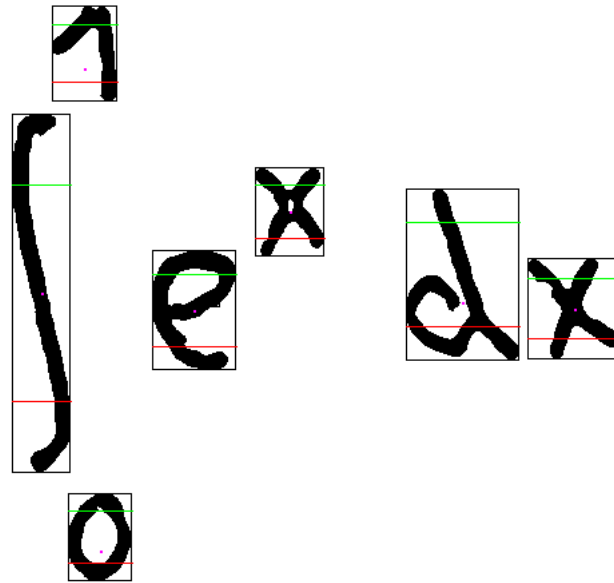
O próximo passo é definir os limiares do centroide. Novamente os limiares superior e inferior podem ser definidos conforme a grafia do símbolo. Porém, neste trabalho todas as grafias tiveram melhores resultados com os mesmos limiares, e portanto, os valores não ficaram iguais aos de Tapia (2005). Veja a tabela 4.3.

O resultado final da definição dos centroides e limiares pode ser observado na figura 4.8. Esses valores encontrados são úteis para analisar a dominância entre os símbolos.

4.5.1 Identificação das barras horizontais

Na etapa de treinamento, o símbolo: “—”, foi treinado para ser reconhecido como barra horizontal. Porém, esse símbolo pode ser representado pela barra horizontal da fração, pelo sinal de subtração, ou ainda, dois traços horizontais próximos, podem representar o sinal de igual (=). Este último símbolo não é segmentado corretamente pelo algoritmo de componentes conexos, e por isso, é necessário juntar as duas barras horizontais para formar o sinal de igual. Antes de começar a construir a BST, é necessário analisar todas as barras horizontais da expressão e classificá-las como subtração, fração ou sinal de igualdade. Além de identificar as barras horizontais, Tapia (2005) mudou o *bounding-box* das frações para corrigir alguns problemas causados na construção da BST proposta por Zanibbi, Blostein e Cordy (2001). Neste

Figura 4.8: Centroides e limiares dos símbolos



Produzido pelo autor.

trabalho além de corrigir o *bounding-box* e recalcular os centroides e limiares para a fração, é feito o mesmo para o sinal de igualdade. O pseudocódigo poder ser visto na seção B.2 dos anexos.

4.5.2 Grafo e Árvore Geradora Mínima

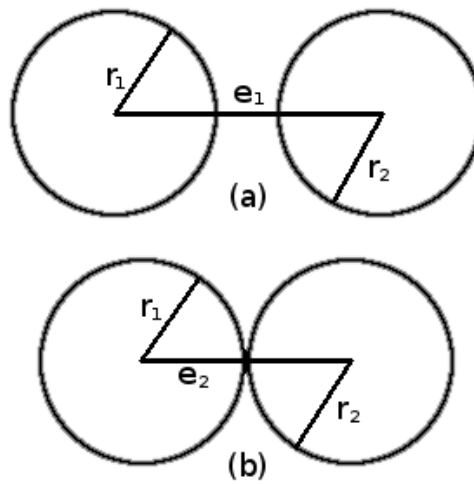
Após definir os centroides e limiares, Tapia (2005) sugeriu a utilização de árvore geradora mínima para a correção de problemas que podem acontecer durante a construção da árvore de *baselines* (BST). Antes de iniciar a construção da BST, foi criado um grafo totalmente conectado, e depois encontrado a árvore geradora mínima (MST). A construção do grafo foi implementada de maneira diferente dos outros autores.

Cada símbolo da expressão matemática é um vértice do grafo. Para cada vértice, foram criadas arestas ligando o vértice de partida com todos os vértices do grafo. Dessa maneira, o grafo ficou totalmente conectado, inclusive, com arestas paralelas para cada par de vértices do grafo. Por isso, foi necessário remover as arestas repetidas, eliminando as arestas paralelas do grafo.

O peso de cada aresta foi definido de maneira diferente do trabalho de Tapia (2005). Tapia (2005) sugeriu distribuir os pesos pela distância euclidiana entre os centroides de dois

símbolos. Porém, se um símbolo domina outro, o peso deverá ser a menor distância entre pontos de atração existentes no *bounding-box* dos dois símbolos. Este trabalho, não utilizou pontos de atração. Como o objetivo é diminuir o peso das arestas em caso de dominância, uma maneira menos custosa de calcular os pesos, é utilizar uma técnica de detecção baseada em colisão. A figura 4.9 mostra um exemplo de colisão de círculos. As variáveis r_1 e r_2 são os raios dos dois círculos, e as variáveis e_1 e e_2 são as distâncias euclidianas entre os pontos centrais dos círculos. No caso (a), os círculos estão afastados, e por isso, $e_1 > r_1 + r_2$. No caso (b), os círculos estão encostados, e por isso, $e_2 = r_1 + r_2$. Caso $e_1 < r_1 + r_2$ acontecerá colisão e um círculo ficará por cima do outro.

Figura 4.9: Colisão de círculos.



Produzido pelo autor.

O mesmo conceito de colisão de círculos pode ser adaptado para calcular o peso das arestas entre dois vértices do grafo. Se um símbolo não domina outro, o peso da aresta será a distância euclidiana entre os centroides, se um símbolo domina outro, o peso será, de forma aproximada, a menor distância entre os dois símbolos. A seguir o pseudocódigo do cálculo dos

pesos das arestas do grafo.

Algoritmo 2: PESO ARESTAS

Entrada: símbolo origem (s_1) e símbolo destino (s_2)

Saída: Peso entre o vértice de origem e vértice de destino

```

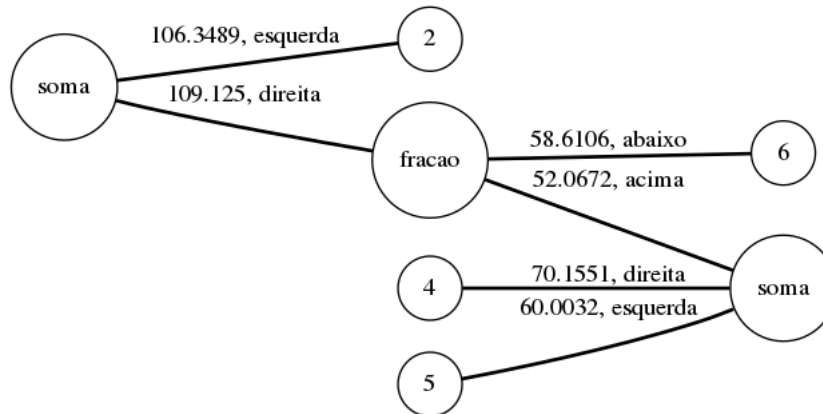
1  início
2       $diff\_centroid\_x = s_1.centroid\_x - s_2.centroid\_x;$ 
3       $diff\_centroid\_y = s_1.centroid\_y - s_2.centroid\_y;$ 
4       $dist\_euclidiana = \sqrt{(diff\_centroid\_x)^2 + (diff\_centroid\_y)^2};$ 
5       $desconto = 0;$ 
6      se domina( $s_1, s_2$ ) então
7          se  $abs(diff\_centroid\_x) > abs(diff\_centroid\_y)$  então
8               $desconto = (s_1\_largura + s_2\_largura)/2;$ 
9          senão
10              $desconto = (s_1\_altura + s_2\_altura)/2;$ 
11         fim
12     fim
13 fim
14 retorna  $dist\_euclidiana - desconto;$ 

```

Após construir o grafo, foi aplicado o algoritmo de Kruskal para encontrar a árvore geradora mínima (MST). O algoritmo de Prim apresentado na seção 2.12 também pode ser usado para encontrar a MST.

A figura 4.10 mostra a MST gerada pelo sistema da expressão $2 + \frac{4+5}{6}$.

Figura 4.10: MST gerada pelo sistema.



Produzido pelo autor.

4.5.3 Árvore de *Baselines*

A construção da árvore de *baselines* é um algoritmo recursivo que se divide em três partes:

1. Encontrar a *baseline* principal a partir de uma lista de símbolos;

2. Encontrar as demais *baselines* que estão associadas com a principal;
3. Aplicar o algoritmo recursivamente para cada nova *baseline* encontrada;

O pseudocódigo a seguir resume as etapas da construção da árvore, lembrando que a recursão acontece somente se existir uma *baseline* na região a ser explorada.

Algoritmo 3: CONSTRÓI ÁRVORE DE BASELINES

Entrada: lista de símbolos
Saída: lista de símbolos

```

1 Função constroi_arvore_baselines(simbolos) início
2   baseline = constroi_baseline_principal(simbolos[0]);
3   associa_pela_mst(baseline);
4   para cada símbolo da baseline faça
5     constroi_arvore_baselines(simbolo.acima);
6     constroi_arvore_baselines(simbolo.abaixo);
7     constroi_arvore_baselines(simbolo.sobrescrito);
8     constroi_arvore_baselines(simbolo.subescrito);
9     constroi_arvore_baselines(simbolo.subexpressao);
10  fim
11  retorna baseline;
12 fim
```

O algoritmo para encontrar a *baseline* principal a partir de um conjunto de símbolos começa pelo símbolo mais à esquerda do conjunto. A partir deste, é selecionado o primeiro símbolo à direita, e aplicado o algoritmo recursivamente neste novo símbolo. O Algoritmo 3 detalha a construção da *baseline* principal.

Algoritmo 4: CONSTRÓI BASELINE PRINCIPAL

Entrada: um símbolo
Saída: *baseline*

```

1 Função constroi_baseline_principal(simbolo) início
2   baseline = [ ];
3   se simbolo nao existir então
4     retorna baseline;
5   fim
6   direita = primeiro símbolo a direita do símbolo passado como parâmetro;
7   vizinhos = [ ];
8   vizinhos.inserir(constroi_baseline_principal(direita));
9   baseline.inserir(simbolo);
10  baseline.inserir(vizinhos);
11  retorna baseline;
12 fim
```

A definição de **direita** pode ser implementada de várias formas. Uma das formas é selecionar o primeiro símbolo que possuir a coordenada x do centroide maior que a coordenada $x + largura$ do *bounding-box* do símbolo atual. É recomendável limitar a coordenada y pelo

mesmo y e *altura* do símbolo atual, ou ainda, limitar o y pelos limiares superior e inferior. Este trabalho utiliza a primeira opção por apresentar melhores resultados.

O pseudocódigo para associar as *baselines* pela MST se encontra aos anexos B.3 deste trabalho.

4.5.4 Conversão para código LaTeX

Após ter sido criada a BST, para obter o código em LaTeX foi implementada uma função recursiva que percorre a BST, adicionando os símbolos de cada *baseline* em uma mesma *string*, e para os símbolos que possuem baselines nas regiões acima, abaixo, subescrito, sobrescrito e subexpressão, o algoritmo é chamado recursivamente em cada região. Ao final, a *string* é retornada. Esta *string* (código LaTeX) é retornada ao cliente (navegador) onde renderiza a expressão matemática utilizando a biblioteca *javascript* MathJAX.

4.6 Escopo

As expressões matemáticas que o sistema pode reconhecer são:

- Aritméticas básicas (soma, subtração, multiplicação e divisão)
- Polinômios
- Funções Racionais (divisão de polinômios)
- Funções Exponenciais
- Somatórios
- Integrais

Expressões matemáticas que o sistema não reconhece corretamente:

- Funções Trigonométricas
- Números com casas decimais (com vírgulas e pontos)
- Raizes

Outras restrições:

- A região da imagem selecionada (*crop*) para o reconhecimento deve ter apenas uma expressão matemática.
- A escrita da expressão deve ser em folhas que não possuam linhas, ou seja, folhas de caderno não são aceitas.
- A expressão deve ser escrita em uma única linha, ou seja, não pode haver quebra da expressão e continuar na próxima linha.
- Todos os símbolos da expressão devem estar separados. A Figura 4.11 é um exemplo do que não é permitido. Os símbolos não podem estar sobrepostos. O mesmo vale para *sin*, *cos*, etc.

Figura 4.11: Exemplo de símbolos sobrepostos



Produzido pelo autor.

- A expressão deve ser escrita de forma alinhada na horizontal (ver a Figura 3.1 de *skew correction* nos trabalhos relacionados).

4.7 Implementação

A aplicação *web* foi desenvolvida na linguagem Ruby utilizando o *framework* Sinatra. O sistema reconhecedor de expressões foi desenvolvido em três linguagens. As etapas de pré-processamento, segmentação e pós-segmentação foram desenvolvidas em C++ utilizando a biblioteca OpenCV. A etapa de treinamento e classificação dos símbolos foi desenvolvida em Python com as bibliotecas Keras e TensorFlow. A etapa de análise estrutural foi desenvolvida em Ruby. A figura 4.12 é a imagem da página inicial do sistema.

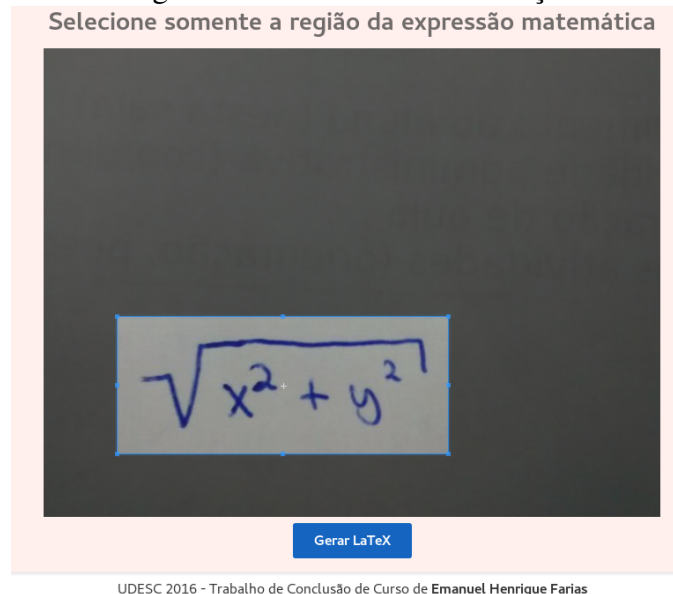
Após realizar o *upload* da imagem, o sistema possui uma opção para selecionar a região da imagem em que se encontra a expressão matemática. Para implementar a ferramenta de seleção foi utilizada a biblioteca Cropper (figura 4.13).

Figura 4.12: Aplicação web



Produzido pelo autor.

Figura 4.13: Ferramenta de seleção

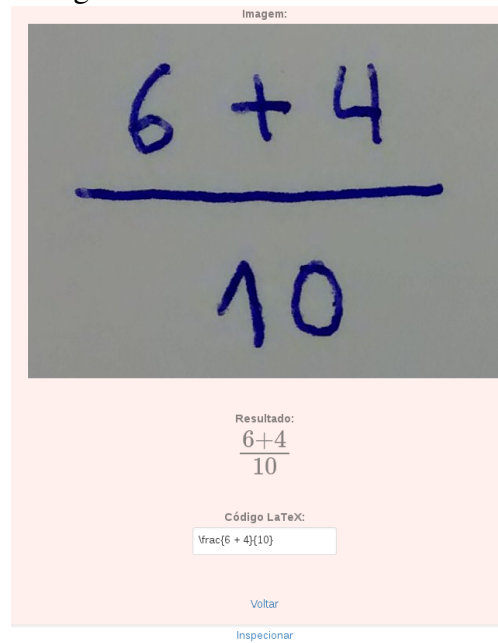


Produzido pelo autor.

A página do resultado do sistema mostra a imagem original, o resultado da expressão renderizado na tela pela biblioteca MathJax, e o código LaTeX para o usuário copiar (figura 4.14).

Além de mostrar o resultado final, o sistema possui uma opção para inspecionar as etapas do reconhecimento, veja a figura 4.15.

Figura 4.14: Resultado do sistema



Produzido pelo autor.

Figura 4.15: Inspeccionar etapas do reconhecimento

Pré Processamento	Segmentação	Pós Segmentação	Reconhecimento	Centroides e Limiares	Árvore Geradora Mínima
			—		fracao
			2		2
			+		soma
			7		7
			5		5
			1		subtracao
			4		4

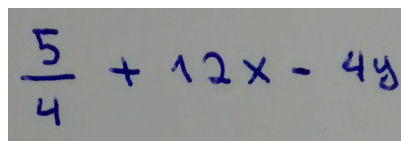
Produzido pelo autor.

5 Resultados

Durante o desenvolvimento do sistema várias expressões criadas pelo autor deste trabalho foram testadas e conseguiram ser reconhecidas com sucesso. Porém, foram coletadas expressões matemáticas de seis pessoas e pode-se identificar vários problemas.

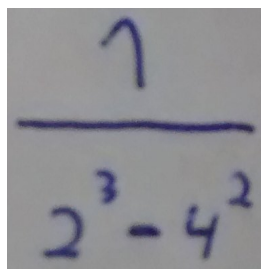
5.1 Expressões reconhecidas

Expressões escritas de forma legível, alinhada horizontalmente e com todos os símbolos separados puderam ser reconhecidas com sucesso. A seguir alguns exemplos de expressões que foram reconhecidas corretamente (figuras 5.1 a 5.5).



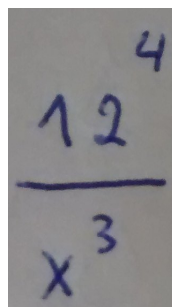
Resultado: $\frac{5}{4} + 12x - 4y$

Figura 5.1: Expressão básica



Resultado: $\frac{1}{2^3 - 4^2}$

Figura 5.2: Denominador com subtração



Resultado: $\frac{12^4}{x^3}$

Figura 5.3: Fração com expoentes

Resultado: $\int_0^5 \int_0^{x^2+1} x^3 dx$

Figura 5.4: Integral dupla

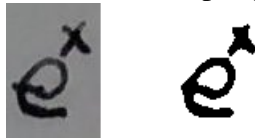
Resultado: $x^{2^3} + 12^4$

Figura 5.5: Duplo expoente

5.2 Problemas Encontrados

Problemas encontrados em uma etapa se propagam para as etapas seguintes. Por exemplo, a primeira etapa (pré-processamento) pode modificar os traços dos símbolos fazendo com que a etapa do reconhecimento não classifique corretamente. No exemplo da figura 5.6, a etapa de pré-processamento deformou o símbolo x , fazendo que a etapa de reconhecimento o classificasse como 2, e por isso, o resultado final ficou e^2 , porém o correto é e^x . A combinação de filtro de mediana com limiarização adaptativa pode causar este tipo de problema.

Figura 5.6: Problema com o pré-processamento

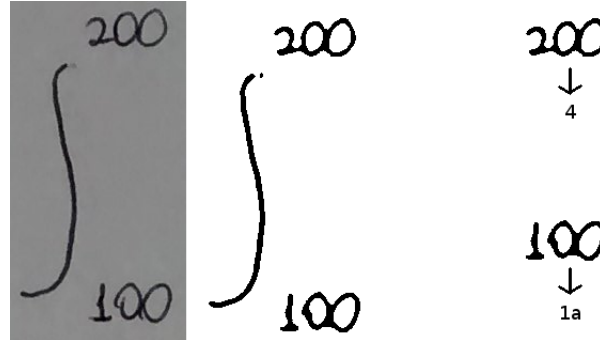


Produzido pelo autor.

Problemas da etapa de segmentação são frequentes quando dois ou mais símbolos estão justapostos. Este problema já era previsto devido ao método que foi utilizado para segmentar (componentes conexos) retornar vários símbolos como se fossem apenas um. Ao realizar os testes com os participantes, foi informado para que todos os símbolos fossem escritos de maneira separada. Porém, duas das seis pessoas acabaram escrevendo símbolos justapostos.

A figura 5.7 mostra os limitantes da integral justapostos, e por isso, foram classificados errados. O lado direito da imagem mostra como os símbolos foram classificados. O resultado final foi $\int_{1a}^4$.

Figura 5.7: Problema com o segmentação de símbolos justapostos



Produzido pelo autor.

A etapa pós-segmentação redimensiona e trata os símbolos antes de serem classificados pela rede neural. Nesta etapa o símbolo da raiz é muito modificado, tornando-o quase imperceptível inclusive para os seres humanos. A imagem 5.8 mostra o símbolo da raiz antes e depois da pós-segmentação. Para o classificador conseguir classificar corretamente este símbolo, foi necessário adicionar na base de dados o símbolo da raiz modificado e retreinar a rede neural. Desta forma, a rede neural consegue classificar corretamente. Como o número de classes treinadas foi pequeno e não tinha nenhum símbolo parecido com este, esta solução pôde ser usada, porém, se o número de classes for grande, as chances de haver algum símbolo muito parecido com esse modificado aumentam, e por isso, essa solução não é a ideal.

Figura 5.8: Problema com a raiz

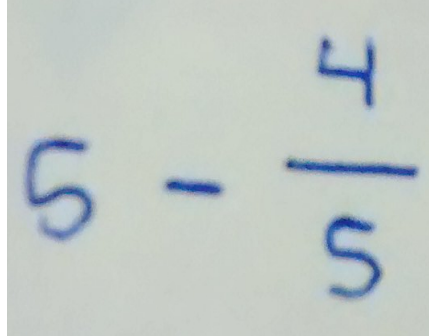


Produzido pelo autor.

A etapa de classificação dos símbolos depende muito da base de dados utilizada. No geral o classificador reconheceu bem os símbolos que foram treinados, porém, para que o sistema de reconhecimento de expressões possa ser utilizado na prática, deverá ser adicionado mais símbolos matemáticos (letras gregas) e o restante do alfabeto latino. Adicionar mais símbolos pode gerar conflitos com símbolos já existentes (por exemplo o s e o 5), e por isso, requer

um número muito maior de todos os símbolos do conjunto. Alguns problemas aconteceram devido os símbolos serem parecidos, veja a figura 5.9. O classificador interpretou 6 no lugar do 5, o resultado final foi: $6 - \frac{4}{5}$. Para diminuir esse tipo de problema é necessária uma base de dados maior e com os símbolos escritos por muitas pessoas diferentes.

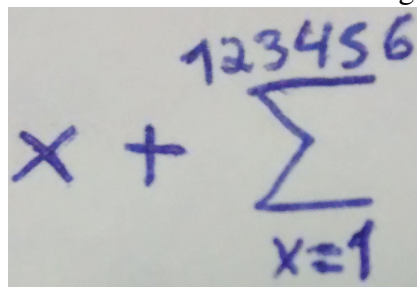
Figura 5.9: Problema com o classificador



Produzido pelo autor.

A etapa da análise estrutural apresentou diversos problemas. O primeiro problema é quando símbolos que não estão na região de dominância, deviam ser dominados. Veja a figura 5.10. O problema com os argumentos do somatório é que saem da região acima do *bounding-box*. Os símbolos 1, 2 e 6 não estão na região acima do somatório. Para resolver o problema do 6, basta selecionar os símbolos que possuem aresta na MST com os símbolos que estão na *baseline* acima. Porém, essa estratégia não resolve para os símbolos 1 e 2 pois a análise das *baselines* é sempre da esquerda para direita, e o primeiro símbolo da esquerda da região acima é o 3. O resultado final do sistema foi: $x + \sum_{x=1}^{3456}$.

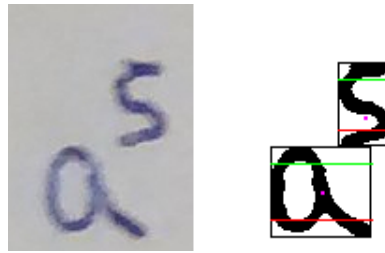
Figura 5.10: Problema com símbolos fora da região de dominância



Produzido pelo autor.

Outro problema que acontece na análise estrutural é quando um símbolo deve estar na região de sobrescrito, porém, o centroide está na região acima. Na figura 5.11, pode-se observar que o centroide do símbolo 5 está acima do *bounding-box* do símbolo *a*. Este problema acontece raramente, e quando acontece, é devido ao símbolo ter se alongado horizontalmente. No caso da figura 5.11, a letra *a* se alongou à direita.

Figura 5.11: Problema com centroides de símbolos em regiões erradas



Produzido pelo autor.

Outro problema ocorrido na etapa da análise estrutural foi o reconhecimento de subexpressões. Na implementação deste trabalho o único símbolo que possui subexpressões é a raiz. O problema acontece devido ao centroide da raiz estar localizado na própria região de subexpressão. A figura 5.12 mostra os centroides e *bounding-boxes* da subexpressão. O resultado final foi $\sqrt{x+y^2}$. A maneira que a árvore de *baselines* foi implementada, cada nó da árvore guarda os filhos, e não o pai. Para resolver este problema o sistema deve ser modificado para cada nó guardar a informação do nó pai. Desta forma, é possível retirar o símbolo raiz da *baseline* de subexpressões eliminando problemas que ocorrem na região de subexpressão.

Figura 5.12: Problema com a região de subexpressão



Produzido pelo autor.

6 Considerações Finais

Os sistemas de reconhecimento de expressões matemáticas possuem várias utilidades desde facilitar a entrada de expressões no computador via código LaTeX até auxiliar deficientes visuais pela leitura de documentos e papéis com fórmulas matemáticas (SIDER; OZAHATA; HIRATA, 2010).

As principais etapas para desenvolver um reconhecedor de expressões matemáticas são: segmentação, classificação e análise estrutural. Este trabalho apresenta um estudo das etapas de reconhecimento de expressões manuscritas e possíveis algoritmos utilizados em cada uma das etapas. O objetivo desse estudo serviu para a criação de uma aplicação *web* capaz de reconhecer expressões matemáticas manuscritas a partir de uma imagem e como resultado a geração do código LaTeX.

Na etapa de pré-processamento, a imagem é convertida para tons de cinza, filtro mediana é aplicado para a redução de ruídos e depois é aplicado a limiarização para gerar uma imagem binária. Na etapa da segmentação é aplicado o método de componentes conexos. Este método não segmenta corretamente alguns tipos de símbolos, porém, na etapa da análise estrutural, o sinal de igualdade é corrigido.

A próxima etapa do reconhecimento é a classificação. O método utilizado foi redes neurais convolucionais. Na literatura não foi encontrado nenhum trabalho utilizando este classificador para reconhecer símbolos matemáticos. O treinamento da rede foi feito com quatro bases de dados, uma do próprio autor deste texto, e três de autores distintos. As bases de dados dos outros autores tiveram que ser tratadas e redimensionadas para o tamanho 32×32 .

A etapa seguinte é a da análise estrutural que tem como objetivo construir a árvore que representa a expressão matemática. Os métodos utilizados foram BST e MST. Estes métodos foram baseados nos trabalhos de Zanibbi, Blostein e Cordy (2001) e Tapia (2005). A implementação não seguiu exatamente como foi descritos nos trabalhos dos autores. Muitos parâmetros tiveram que ser definidos empiricamente. Algumas melhorias foram feitas que podem afetar no desempenho do sistema. Uma delas é o cálculo dos pesos do grafo que possui complexidade de tempo constante, e no trabalho de Tapia (2005) o cálculo dos pesos possui complexidade quadrática. Outro ajuste feito é a criação de uma única MST antes da construção

da BST, enquanto que o trabalho de Tapia cria MSTs em cada recursão da criação da BST. Com este ajuste, ainda é preciso validar se os sistemas conseguem reconhecer as mesmas expressões.

Dados foram coletados de seis pessoas e com isso foi possível observar vários problemas de reconhecimento. Esses problemas foram causados pelo mau alinhamento da expressão, símbolos não treinados na rede neural, símbolos justapostos, entre outros. Porém, pode-se observar que expressões escritas de maneira bem alinhada horizontalmente, legíveis e com todos os símbolos separados puderam ser reconhecidas com sucesso.

Este foi um trabalho completo com a implementação de todas as etapas de um reconhecedor *offline* de expressões matemáticas. O trabalho apresentou bons resultados, mesmo utilizando métodos comuns na área de processamento de imagens. Para implementar um sistema mais robusto, todas as etapas podem ser melhoradas com o uso de métodos mais avançados, como os apresentados a seguir.

6.1 Trabalhos Futuros

Na primeira etapa (pré-processamento), cujo objetivo é eliminar ruídos e separar o fundo da escrita, foi implementada com um filtro e uma simples limiarização. Problemas com essa estratégia acontecem quando a pessoa que está escrevendo não posiciona a caneta sobre o papel, fazendo com que o traço seja muito fraco. Outro problema é que o filtro aplicado pode deformar o traçado. A redução do tamanho máscara do filtro pode ser uma solução. Estratégias mais avançadas para tratar documentos utilizam *clustering* para separar o fundo da escrita. Considerando que a maior parte da imagem é composta pelo fundo e a menor parte pela escrita, é possível formar dois grupos pelas cores da imagem (ZUCKER, 2016).

A etapa de segmentação foi implementada com o algoritmo de componentes conexos, e por isso, não segmenta corretamente símbolos justapostos nem símbolos compostos por vários componentes desconexos. Uma forma de tentar resolver esses problemas é a segmentação por projeção de histogramas. Okamoto e Miao (1991) propôs particionar uma expressão matemática aplicando recursivamente o método *Projection Profile Cutting* que utiliza projeção de histogramas nos eixos horizontal e vertical conseguindo segmentar os símbolos mesmo estando justapostos. Uma forma para resolver o problema de um símbolo compostos por vários componentes é a utilização redes neurais específicas para detecção e segmentação dos símbolos.

A etapa de classificação dos símbolos pode ser melhorada com o aumento da base

de dados. Para que o trabalho reconheça mais tipos de expressões, é necessário adicionar todo o alfabeto latino e grego. Além disso, aumentar a quantidade de caligrafias distintas na *dataset* é muito importante para que a rede consiga reconhecer diversos estilos de escrita. Outra estratégia para melhorar a classificação é treinar as letras cursivas e letras de fôrma em classes diferentes.

Muitos dos problemas da análise estrutural acontecem devido aos limiares dos símbolos serem padrões para cada grafia. O ideal é que cada símbolo tivesse seus próprios limiares. Além dos limiares inferior e superior, adicionar limiares na esquerda e na direita poderia ajudar a resolver alguns problemas. Outra forma de melhorar a análise estrutural é criar uma classe comportamental para cada símbolo. Na implementação deste trabalho, símbolos parecidos pertencem a mesma classe comportamental, por exemplo, todos os símbolos do alfabeto pertencem a classe comportamental das letras.

Outra forma de melhorar os resultados é a implementação de uma etapa de correção de erros pós análise estrutural. Erros comuns na etapa de reconhecimento podem ser resolvidos analisando o contexto em que os símbolos aparecem. Por exemplo, símbolos como “(”, deverão ser fechados com “)”. O sinal “+” espera pelo menos um argumento em sua direita, o mesmo acontece com o sinal “-”. O símbolo raiz sempre possui uma subexpressão, mas o índice é opcional.

Bibliografia

- ANDERSON, R. H. Syntax-directed recognition of hand-printed two-dimensional mathematics. In: ACM. *Symposium on Interactive Systems for Experimental Applied Mathematics: Proceedings of the Association for Computing Machinery Inc. Symposium*. 1967. p. 436–459.
- AUSBROOKS, R. et al. Mathematical markup language (mathml) version 2.0 . w3c recommendation. *World Wide Web Consortium*, Boston, MA, USA:, v. 2003, 2003.
- BAGE, D. D.; ADHIYA, K.; GHARDE, S. S. A new approach for recognizing offline handwritten mathematical symbols using character geometry. *International Journal Of Innovative Research In Science, Engineering And Technology*, v. 2, July 2010.
- BHARAMBE, M. Article: Recognition of offline handwritten mathematical expressions. *IJCA Proceedings on National conference on Digital Image and Signal Processing*, DISP 2015, n. 2, p. 35–39, April 2015.
- CAMPOS, T. E. de; BABU, B. R.; VARMA, M. Character recognition in natural images. In: *VISAPP (2)*. 2009. p. 273–280.
- CHAJRI, Y.; BOUIKHALENE, B. Handwritten mathematical symbols dataset. *Data in brief*, Elsevier, v. 7, p. 432–436, 2016.
- CHAN, K.-F.; YEUNG, D.-Y. Mathematical expression recognition: a survey. *International Journal on Document Analysis and Recognition*, Springer, v. 3, n. 1, p. 3–15, 2000.
- CHANG, S.-K. A method for the structural analysis of two-dimensional mathematical expressions. *Information Sciences*, Elsevier, v. 2, n. 3, p. 253–272, 1970.
- CHOU, P. A. Recognition of equations using a two-dimensional stochastic context-free grammar. In: INTERNATIONAL SOCIETY FOR OPTICS AND PHOTONICS. *1989 Advances in Intelligent Robotics Systems Conference*. 1989. p. 852–865.
- COSTAGLIOLA, G. et al. A parsing methodology for the implementation of visual systems. *Software Engineering, IEEE Transactions on*, IEEE, v. 23, n. 12, p. 777–799, 1997.

- DESAI, A. A. Segmentation of characters from old typewritten documents using radon transform. *International Journal of Computer Applications (0975 8887) Volume*, v. 37, 2012.
- DOUGHERTY, E. R.; LOTUFO, R. A. *Hands-On Morphological Image Processing*. : SPIE Press, 2003. ISBN 0-8194-4720-X.
- DUNNE, R. A.; CAMPBELL, N. A. On the pairing of the softmax activation and cross-entropy penalty functions and the derivation of the softmax activation function. In: CITESEER. *Proc. 8th Aust. Conf. on the Neural Networks, Melbourne, 181*. 1997. v. 185.
- FAURE, C.; WANG, Z. X. Automatic perception of the structure of handwritten mathematical expressions. *Computer processing of handwriting*, World Scientific, p. 337–361, 1990.
- FERNANDES, B. J. T. Redes neurais com extração implícita de características para reconhecimento de padrões visuais. Universidade Federal de Pernambuco, p. 24–31, 2013.
- GARAIN, U.; CHAUDHURI, B. B. Segmentation of touching symbols for ocr of printed mathematical expressions: An approach based on multifactorial analysis. In: *Proceedings of the Eighth International Conference on Document Analysis and Recognition*. Washington, DC, USA: IEEE Computer Society, 2005. (ICDAR '05), p. 177–181. ISBN 0-7695-2420-6. Disponível em: <<http://dx.doi.org/10.1109/ICDAR.2005.212>>.
- GERSTING, J. L. *Fundamentos matemáticos para a ciência da computação: um tratamento moderno de matemática discreta*. : Livros Técnicos e Científicos, 2004.
- GLOROT, X.; BORDES, A.; BENGIO, Y. Deep sparse rectifier neural networks. In: *Aistats*. 2011. v. 15, p. 275.
- GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing (3rd Edition)*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc., 2006. ISBN 013168728X.
- GRBAVEC, A.; BLOSTEIN, D. Mathematics recognition using graph rewriting. In: IEEE. *Document Analysis and Recognition, 1995., Proceedings of the Third International Conference on*. 1995. v. 1, p. 417–421.
- JAKJOURD, W.; LAZREK, A. Segmentation method of offline mathematical symbols. In: IEEE. *Multimedia Computing and Systems (ICMCS), 2011 International Conference on*. 2011. p. 1–7.

KINGMA, D.; BA, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

KIRSCH, D. *Detexify: Erkennung handgemalter LaTeX-symbole*. Tese (Doutorado) — PhD thesis, Westfälische Wilhelms-Universität Münster, 2010.

KUNCHEVA, L. I. *Combining pattern classifiers: methods and algorithms*. : John Wiley & Sons, 2004.

LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, IEEE, v. 86, n. 11, p. 2278–2324, 1998.

LEE, H.-J.; LEE, M.-C. Understanding mathematical expressions using procedure-oriented transformation. *Pattern Recognition*, Elsevier, v. 27, n. 3, p. 447–457, 1994.

MARINAI, S.; FUJISAWA, H. *Machine learning in document analysis and recognition*. : Springer, 2007. 277 p.

MATSAKIS, N. Recognition of handwritten mathematical expressions. *Master's thesis, Massachusetts Institute of Technology, Cambridge, MA*, p. 1–59, 1999.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, Springer, v. 5, n. 4, p. 115–133, 1943.

MUÑOZ, F. Á. *Off-line Recognition of Printed Mathematical Expressions Using Stochastic Context-Free Grammars*. Tese (Doutorado), 2011. Acessado em 14 Outubro 2015. Disponível em: <<https://riunet.upv.es/bitstream/handle/10251/13732/Tesina.pdf?sequence=1>>.

MUÑOZ, F. Á. *Mathematical Expression Recognition based on Probabilistic Grammars*. 277 p. Tese (Doutorado), 2015. Acessado em 15 Agosto 2015. Disponível em: <<https://riunet.upv.es/handle/10251/51665>>.

MURPHY, K. P. *Machine learning: a probabilistic perspective*. : MIT press, 2012.

NAIK, S.; METKEWAR, P. Recognizing offline handwritten mathematical expressions (me) based on a predictive approach of segmentation using k-nn classification. *International Journal of Technology*, v. 6, p. 345–354, 2015.

NIELSEN, M. A. Neural networks and deep learning. *URL*: <http://neuralnetworksanddeeplearning.com> (visitado em 28.05.2015), 2015.

- NIXON, M. *Feature extraction & image processing*. 2 ed. : Academic Press, 2008. 281–284 p.
- OKAMOTO, M.; MIAO, B. Recognition of mathematical expressions by using the layout structures of symbols. In: *Proceedings of the First International Conference on Document Analysis and Recognition*. 1991. v. 1, p. 242–250.
- PEDRINI, H.; SCHWARTZ, W. *Análise de imagens digitais: princípios, algoritmos e aplicações*. : THOMSON PIONEIRA, 2008. ISBN 9788522105953.
- PHILLIPS, I. T. Methodologies for using uw databases for ocr and image-understanding systems. In: INTERNATIONAL SOCIETY FOR OPTICS AND PHOTONICS. *Photonics West'98 Electronic Imaging*. 1998. p. 112–127.
- QUINIOU, S. et al. Hamex-a handwritten and audio dataset of mathematical expressions. In: IEEE. *Document Analysis and Recognition (ICDAR), 2011 International Conference on*. 2011. p. 452–456.
- SIDER, R.; HIRATA, N. S. Análise estrutural de expressões matemáticas manuscritas usando baselines e árvore geradora mínima. p. 4–5, Dezembro 2015. Acessado em 15 Agosto 2015. Disponível em: <<http://www.ime.usp.br/cef/mac499-09/monografias/bruno-ricardo/monografia/monografia-ricardo.pdf>>.
- SIDER, R.; OZAHATA, B. Y.; HIRATA, N. S. Expressmath: Análise estrutural de expressões matemáticas manuscritas. *Iniciação científica e tecnológica: o jovem pesquisador em ação II*, p. 109, 2010.
- SMITHIES, S. *Freehand formula entry system*. Tese (Doutorado) — University of Otago, 1999.
- SRIVASTAVA, N. et al. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, v. 15, n. 1, p. 1929–1958, 2014.
- STRIA, J. et al. Mfrdb: Database of annotated on-line mathematical formulae. In: IEEE. *Frontiers in Handwriting Recognition (ICFHR), 2012 International Conference on*. 2012. p. 542–547.
- TAPIA, E. *Understanding mathematics: A system for the recognition of on-line handwritten mathematical expressions*. Tese (Doutorado) — Freie Universität Berlin, Germany, 2005.
- TAPIA, E. A survey on recognition of on-line handwritten mathematical notation. p. 3, 2007.

THOMA, M. On-line recognition of handwritten mathematical symbols. *arXiv preprint arXiv:1511.09030*, 2015.

VAMVAKAS, G. et al. Hybrid off-line ocr for isolated handwritten greek characters. In: *The Fourth IASTED International Conference on Signal Processing, Pattern Recognition, and Applications (SPPRA 2007)*. 2007. p. 197–202.

VAPNIK, V. N.; VAPNIK, V. *Statistical learning theory*. : Wiley New York, 1998.

WARD, J. et al. Efficient mapping of the training of convolutional neural networks to a cuda-based cluster. *Eindhoven University of Technology, The Netherlands, Citeseer*, v. 12, 2011.

ZANIBBI, R.; BLOSTEIN, D.; CORDY, J. R. Baseline structure analysis of handwritten mathematics notation. In: IEEE. *Document Analysis and Recognition, 2001. Proceedings. Sixth International Conference on*. 2001. p. 768–773.

ZANIBBI, R.; BLOSTEIN, D.; CORDY, J. R. Recognizing mathematical expressions using tree transformation. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, IEEE, v. 24, n. 11, p. 1455–1467, 2002.

ZANIBBI, R. L. A. et al. *Recognition of mathematics notation via computer using baseline structure*. : Citeseer, 2001.

ZUCKER, M. *Compressing and enhancing hand-written notes*. 2016. <https://mzucker.github.io/2016/09/20/noteshrink.html>. Acessado: 14 de novembro de 2016.

A Apêndice: *Datasets*

A seguir são listadas *datasets* publicadas e que podem ser utilizadas para pesquisas. Para cada uma são detalhados o tipo (*online* ou *offline*), manuscrito ou impresso, o formato da informação, aonde encontrar os dados e a descrição.

- detexify-data (KIRSCH, 2010)

Tipo: *Online*/manuscrito.

Formato: Formato YAML ($x, y, time$). Pontos espaçados com dados temporais.

Licença: ODbL (ODC *Open Database License*).

Tamanho Aproximado: 136042 símbolos até janeiro de 2015.

Website: <https://github.com/kirel/detexify-data>

Descrição: Os símbolos são dígitos, alfabeto latino minúsculo e maiúsculo, letras gregas e outros símbolos matemáticos.

- HWRT (THOMA, 2015)

Tipo: *Online*/manuscrito.

Formato: Formato YAML ($x, y, timestamp$). Pontos espaçados com dados temporais.

Licença: ODbL (ODC *Open Database License*).

Tamanho Aproximado: 151158 símbolos até janeiro de 2015.

Website: <http://www.martin-thoma.de/write-math/data>

Descrição: Esta *dataset* é composta por 90% da *detexify-data*. Os símbolos são dígitos, alfabeto latino minúsculo e maiúsculo, letras gregas e outros símbolos matemáticos.

- CROHME

Tipo: *Online*/manuscrito.

Formato: Formato InkML. XML modificado que contém as expressões matemáticas em grupos de traços. Cada traço é composto por um conjunto de pontos. Cada ponto tem as coordenadas e o *timestamp*.

Licença: *Creative Commons license* para uso acadêmico. Proibido uso comercial.

Tamanho Aproximado: ?

Website: http://www.isical.ac.in/~crohme/CROHME_data.html

Descrição: *Dataset* popular usada na competição CROHME reconhecimento de expressões matemáticas. Os dados são símbolos e expressões matemáticas inteiras.

●Chars74k (CAMPOS; BABU; VARMA, 2009)

Tipo: *Offline*/manuscrito.

Formato: Imagens.

Licença: ?

Tamanho Aproximado: 74000 símbolos.

Website: <http://www.ee.surrey.ac.uk/CVSSP/demos/chars74k>

Descrição: *Dataset* com 64 classes (dígitos, alfabeto minúsculo e maiúsculo). 7705 símbolos foram obtidos de imagens naturais, 3410 símbolos obtidos via tablet e 62992 foram sintetizados de fontes de computador.

●MfrDB (STRIA, 2012)

Tipo: *Online*/manuscrito.

Formato: XML composto por InkML and MathML.

Licença: Gratuita para uso não comercial.

Tamanho Aproximado: 2018 fórmulas matemáticas até janeiro de 2016.

Website: http://mfr.felk.cvut.cz/Database_download.html

Descrição: 2018 fórmulas matemáticas escritas por 232 participantes por interface gráfica via *mouse*, tela *touch* ou *stylus pen*.

●HAMEX (QUINIOU, 2011)

Tipo: *Online*/manuscrito.

Formato: InkML.

Licença: Não divulgada. Porém, Se entrar em contato eles disponibilizam.

Tamanho Aproximado: 4350 expressões matemáticas.

Website: <http://ivc.univ-nantes.fr/en/databases/HAMEX>

Descrição: Expressões matemáticas manuscritas e faladas (áudio).

●MNIST (LECUN, 1998)

Tipo: *Offline*/manuscrito

Formato: Formato IDX. Armazena vetores e matrizes com valores numéricos que representam a imagem de um dígito.

Licença: *Creative Commons Attribution-Share Alike 3.0*.

Tamanho Aproximado: 60 mil dígitos e mais 10 mil para o conjunto de testes.

Website: <http://yann.lecun.com/exdb/mnist>

Descrição: *Dataset* muito popular de dígitos [0, 9] manuscritos.

●UW-III (PHILLIPS, 1998)

Tipo: *Offline*/impresso.

Formato: formato XFIG.

Licença: Proprietária.

Tamanho Aproximado: 25 páginas impressas de fórmulas matemáticas.

Website: <http://isis-data.science.uva.nl/events/dlia/datasets/uwash3.html>

Descrição: Essa base de dados é paga e custa USD 100.

●(CHAJRI; BOUIKHALENE, 2016)

Tipo: *Offline*/manuscrito.

Formato: Imagens

Licença: ?

Tamanho Aproximado: 10379 símbolos.

Website: <http://www.ncbi.nlm.nih.gov/pmc/articles/PMC4786757/>

Descrição: 10379 símbolos do alfabeto latino, árabe e símbolos matemáticos.

B Apêndice: *Pseudocódigos*

B.1 Pós-segmentação

Algoritmo 5: PÓS-SEGMENTAÇÃO

Entrada: Imagem de um componente

Saída: Imagem de um componente ajustada e redimensionada

```

1  início
2       $lado = \max(largura, altura);$ 
3      se  $lado > 32$  então
4          se  $largura > altura$  então
5               $borda = largura \cdot 0.15;$ 
6              cria nova imagem branca de tamanho  $largura + 2 \cdot borda \times largura + 2 \cdot borda;$ 
7          senão
8               $borda = altura \cdot 0.15;$ 
9              cria nova imagem branca de tamanho  $altura + 2 \cdot borda \times altura + 2 \cdot borda;$ 
10         fim
11         coloca simbolo ao centro da nova imagem;
12         se  $lado > 70$  então
13             dilata a nova imagem;
14         fim
15         redimensiona nova imagem para  $32 \times 32;$ 
16     senão
17         se  $lado > 28$  então
18             cria nova imagem branca de  $38 \times 38;$ 
19             coloca simbolo ao centro da nova imagem;
20             redimensiona nova imagem para  $32 \times 32;$ 
21         senão
22             cria nova imagem em branco de  $32 \times 32;$ 
23             coloca simbolo ao centro da nova imagem;
24         fim
25     fim
26 fim
27 retorna nova imagem

```

B.2 Identificação das Barras Horizontais

Algoritmo 6: IDENTIFICAÇÃO DAS BARRAS HORIZONTAIS

```

1  para cada barra faça
2      Encontra todos os símbolos que barra domina acima e abaixo;
3      se barra dominar algum símbolo abaixo então
4          se existir somente um símbolo abaixo muito próximo então
5              se  $\text{barra}_x$  for próximo a coordenada  $x$  do símbolo abaixo então
6                  se  $\text{barra}_x + \text{barra}_{\text{largura}}$  for próximo a  $x + \text{largura}$  do símbolo abaixo então
7                      atualiza rótulo da barra para igualdade;
8                       $\text{barra}_{\text{altura}} = (\text{simbolo\_abaixo}_y - \text{barra}_y) + \text{simbolo\_abaixo}_{\text{altura}} +$ 
9                           $\text{simbolo\_abaixo}_{\text{largura}}/2 + \text{barra}_{\text{largura}}/2;$ 
10                      $\text{barra}_y = \text{barra}_y - \text{barra}_{\text{largura}}/2;$ 
11                     recalcula centroides e limiares da barra;
12                 fim
13             fim
14         fim
15     se barra não foi rotulada como igualdade (=) então
16         se barra domina símbolos acima e abaixo então
17             atualiza rótulo da barra para fração;
18              $\text{simbolo\_acima} =$  primeiro símbolo acima mais próximo;
19              $\text{simbolo\_abaixo} =$  primeiro símbolo abaixo mais próximo;
20              $\text{barra}_y = \text{simbolo\_acima}_y;$ 
21              $\text{barra}_{\text{altura}} = (\text{simbolo\_abaixo}_y + \text{simbolo\_abaixo}_{\text{altura}}) - \text{barra}_y;$ 
22             recalcula centroides e limiares da barra;
23         senão
24             atualiza rótulo da barra para subtração;
25              $\text{barra}_y = \text{barra}_y - \text{barra}_{\text{largura}}/2;$ 
26              $\text{barra}_{\text{altura}} = \text{barra}_{\text{altura}} + \text{barra}_{\text{largura}};$ 
27             recalcula centroides e limiares da barra;
28         fim
29     fim
30 fim

```

B.3 Associa Baseline pela MST

O pseudocódigo a seguir faz a chamada de algumas funções que não foram definidas neste texto, porém, o nome das funções é sugestivo para sua implementação. Para mais detalhes,

sobre as funções entrar em contato com o autor deste trabalho.

Algoritmo 7: ASSOCIA PELA MST

Entrada: baseline

```

1  Função associa_pela_mst(baseline) início
2      para cada simbolo da baseline faça
3          acima = domina(simbolo, 'acima');
4          abaixo = domina(simbolo, 'abaixo');
5          subescrito = domina(simbolo, 'subescrito');
6          sobrescrito = domina(simbolo, 'sobrescrito');
7          subexpressao = domina(simbolo, 'subexpressao');
8          regioes = [acima, abaixo, subescrito, sobrescrito, subexpressao];
9          para cada regioao das regioes faça
10             candidatos = vizinhos_da_direita(regiao[0]);
11             candidatos.inserir(regiao[0]);
12             Ordena candidatos pela coordenada x (ordem crescente);
13             baseline_regiao = [ ];
14             para cada candidato dos candidatos faça
15                 se mst.verifica_aresta(candidato, simbolo) então
16                     baseline_regiao.inserir(candidato);
17                 fim
18             fim
19             se baseline_regiao.tamanho == 0 então
20                 baseline_regiao.inserir(candidatos[0]);
21             fim
22             i = 0;
23             enquanto i < baseline_regiao.tamanho faça
24                 para cada candidato dos candidatos faça
25                     se mst.verifica_aresta(candidato, baseline_regiao[i]) então
26                         baseline_regiao.inserir(candidato);
27                     fim
28                 fim
29                 i = i + 1;
30             fim
31             simbolo.acima = baseline_regiao se for a regioao atual;
32             simbolo.abaixo = baseline_regiao se for a regioao atual;
33             simbolo.subescrito = baseline_regiao se for a regioao atual;
34             simbolo.sobrescrito = baseline_regiao se for a regioao atual;
35             simbolo.subexpressao = baseline_regiao se for a regioao atual;
36         fim
37     fim
38 fim

```
